

Capitolo 1 – Introduzione all'Intelligenza Artificiale (IA)

1.1 Cos'è l'Intelligenza Artificiale?

Non esiste una definizione unica, ma quattro approcci principali:

Approccio	Descrizione	Focus
Agire umanamente	Test di Turing: un sistema è "intelligente" se simula un essere umano in una conversazione	Comportamento esterno
Pensare umanamente	Modellazione cognitiva: replica dei processi mentali umani (psicologia, neuroscienze)	Funzionamento interno
Pensare razionalmente	Uso della logica per ragionare in modo corretto (leggi del pensiero)	Logica e ragionamento
Agire razionalmente	Agente razionale: agisce per massimizzare il risultato atteso in base alle percezioni	Obiettivi e razionalità

 Il testo adotta l'approccio dell'agire razionalmente.

1.2 I Fondamenti dell'IA L'IA

si basa su più discipline:

- **Filosofia:** logica, razionalità, natura del pensiero (es. Aristotele)
- **Matematica:** logica formale, probabilità, teoria della computazione (es. NP-completezza)
- **Economia:** decisioni ottimali, utilità, teoria dei giochi
- **Neuroscienze:** cervello come elaboratore di informazioni, base delle reti neurali
- **Psicologia:** comportamentismo, cognitivismo → influenza sulla modellazione degli agenti
- **Informatica/ingegneria:** hardware e software, potenza di calcolo (Legge di Moore)
- **Teoria del controllo/cibernetica:** sistemi autoregolanti, feedback (es. Norbert Wiener)

- **Linguistica:** comprensione e produzione del linguaggio → linguistica computazionale

1.3 La Storia dell'IA

Periodo	Eventi principali
1943–1956 (Origini)	Neuroni artificiali (McCulloch & Pitts), Test di Turing
1952–1969 (Entusiasmo iniziale)	Logic Theorist, General Problem Solver, linguaggio Lisp, primi giochi (dama, scacchi)
1969–1986 (Sistemi esperti)	Sistemi a regole, applicazioni specifiche, crisi e "inverno dell'IA"
Dal 1986 (Connessionismo)	Reti neurali + backpropagation
Dal 1987 (IA scientifica)	Metodi statistici, apprendimento automatico, reti bayesiane, HMM
2001/2011 → oggi (Big Data e Deep Learning)	Dati massivi + reti neurali profonde → grandi progressi in visione, voce, linguaggio

1.4 Lo Stato dell'Arte

L'IA è già ampiamente applicata:

- Riconoscimento immagini
 - Traduzione automatica
 - Guida autonoma
 - Diagnosi medica
 - Giochi a livello superumano (scacchi, Go, StarCraft II)
 - Crescita esponenziale della ricerca e delle collaborazioni
-

1.5 Rischi e Opportunità dell'IA

Opportunità:

- Miglioramento in sanità, scienza, economia, ecc.
- Automazione di compiti complessi

Rischi:

- Etica, privacy, sicurezza
- Impatto sul lavoro
- Possibili usi impropri
- Necessità di regolamentazione

👉 **Serve uno sviluppo responsabile e controllato dell'IA.**

Riassunto finale

L'Intelligenza Artificiale è un campo multidisciplinare che mira a creare agenti razionali.

Si basa su diverse discipline (logica, economia, neuroscienze, informatica) e ha vissuto fasi alterne, dagli entusiasmi iniziali ai successi recenti dovuti a deep learning e Big Data.

Ha grandi potenzialità ma anche rischi significativi, che vanno affrontati in modo consapevole.

Cap. 2 – Agenti Intelligenti

Cos'è un agente intelligente?

Un **agente** è qualsiasi entità capace di:

- **Percepire** il proprio ambiente (fisico o virtuale) tramite sensori.
- **Agire** su di esso tramite attuatori.

Il **comportamento** dell'agente deriva da un **programma agente**, che mappa ciò che percepisce alle azioni da compiere.

Questo programma gira su un'**architettura**, ovvero la combinazione hardware/software su cui l'agente è implementato.

La **funzione agente** ha come input l'intera sequenza percettiva.

Razionalità: Agire nel modo giusto

Un **agente razionale** è colui che:

- Fa "la cosa giusta", ovvero sceglie l'azione con le migliori conseguenze attese, in base a:
 - La **sequenza percettiva** ricevuta.

- Le conoscenze disponibili.
- Le azioni che l'agente può effettuare.
- Una **misura di prestazione**, che valuta quanto è buono un certo esito.

La razionalità è ben diversa dall'onniscienza. Un agente ipoteticamente onnisciente conosce già tutte le situazioni possibili e sceglie immediatamente la strada corretta da perseguire. Nella realtà un agente onnisciente non può esistere, in quanto sono sempre presenti fattori ignoti che potrebbero intercorrere.

Concetti associati:

- **Apprendimento**: serve per adattarsi ad ambienti nuovi o mutevoli.
- **Autonomia**: un agente autonomo si affida sempre meno alla conoscenza iniziale e sempre più all'esperienza acquisita.

Natura degli ambienti:

Il modo in cui un ambiente è strutturato condiziona profondamente come progettare un agente. Un ambiente può avere alcune proprietà che lo rendono:

- Completamente o Parzialmente osservabile
- Deterministico o Stocastico
- Episodico o Sequenziale

Inoltre, può essere classificabile come:

- Statico o Dinamico
- Discretezza o continuità
- A Singolo Agente o Multiagente

Struttura degli agenti

Esistono vari **tipi di programmi agente**, a seconda della complessità e dell'obiettivo:

1. **Agenti reattivi semplici**

→ Reagiscono solo all'input attuale, **senza memoria**.

Veloci, ma limitati in ambienti complessi (parzialmente osservabili).

2. **Agenti reattivi basati su modello**

→ Tengono uno **stato interno** del mondo per ricordare la storia.
Più capaci di operare in ambienti dinamici.

3. **Agenti basati su obiettivi (goal-driven)**

→ Hanno uno scopo da raggiungere e pianificano una **sequenza di azioni**.
Richiedono algoritmi di **ricerca e pianificazione**.

4. **Agenti basati sull'utilità (utility-driven)**

→ Scelgono l'azione che **massimizza la desiderabilità** dell'esito. L'utilità viene calcolata attraverso una funzione specifica.
Più flessibili in contesti con **incertezza o obiettivi multipli**.

5. **Agenti capaci di apprendere**

→ Modificano e migliorano il proprio comportamento grazie all'**esperienza**.
Fondamentali per l'**autonomia**.

La parte dell'agente che si occupa dell'apprendimento è composta da:

- Un **critico**, che decide se attivare l'apprendimento o meno.
- Un **modulo di apprendimento**, che modifica la conoscenza dell'agente.
- Un **generatore di problemi** che causa l'esecuzione di azioni esplorative, il cui fine è esporre l'agente a nuove esperienze

Cap. 3 - Risoluzione dei problemi tramite la ricerca

3.1 - Agenti e formulazione dei problemi

- Un **agente risolutore di problemi** decide le azioni in base alle conseguenze future, cercando di raggiungere un obiettivo.
- Il primo passo è la **formulazione dell'obiettivo e del problema**, che richiede di definire:
 - **Spazio degli stati**: tutte le configurazioni possibili dell'ambiente.
 - **Stato iniziale**: la configurazione di partenza.
 - **Azioni (operatori)**: le mosse possibili in ogni stato.
 - **Modello di transizione**: descrive l'effetto di ogni azione in uno stato.
 - **Test obiettivo**: funzione che stabilisce se uno stato è un obiettivo.
 - **Costo del cammino**: assegna un costo numerico a ogni sequenza di azioni.

La **ricerca** consiste nella simulazione delle sequenze di azioni e lo stato raggiunto attraverso queste.

Una **soluzione** è un cammino (sequenza di azioni) che porta dallo stato iniziale a uno stato obiettivo.

Se l'ambiente è **deterministico** viene utilizzato un **sistema ad anello aperto**, ovvero, trovata la soluzione il sistema procede passo per passo senza mantenere le percezioni dell'ambiente. Se invece l'ambiente è **non deterministico**, dopo aver trovato una soluzione potrebbero esserci altri eventi a interrompere il sistema, dunque viene prediletto un **sistema ad anello chiuso**, in cui le percezioni dell'ambiente vengono mantenute e utilizzate per modificare la soluzione al passaggio di stato in stato.

3.2 - Problemi esemplificativi

Problemi semplici e classici sono usati per mostrare la formulazione, come:

a) Il mondo dell'aspirapolvere:

Nel contesto di un problema su **griglia**, il mondo è rappresentato come una matrice bidimensionale di celle quadrate in cui un agente può muoversi tra celle adiacenti (orizzontalmente, verticalmente o diagonalmente). Le celle possono contenere oggetti, ostacoli o essere libere.

Componenti principali del problema:

- **Stati**: descrivono la disposizione degli oggetti nelle celle. Nell'esempio dell'aspirapolvere con 2 celle, ogni cella può essere sporca o pulita, e l'agente può trovarsi in entrambe. Si ottengono 8 stati possibili ($2 \times 2 \times 2$).
- **Stato iniziale**: qualsiasi stato può essere definito come punto di partenza.
- **Azioni**: in un mondo con 2 celle le azioni sono **Sinistra**, **Destra**, **Aspira**. Nei mondi più complessi si possono aggiungere **Su** e **Giù**, o usare azioni **egocentriche** (relative all'agente) come **Avanti**, **Indietro**, **GiraDestra**, **GiraSinistra**.
- **Modello di transizione**: specifica l'effetto di ogni azione. Per esempio:
 - o *Aspira* pulisce la cella corrente,
 - o *Avanti* muove l'agente nella direzione in cui è orientato,
 - o *Indietro* lo muove nella direzione opposta,
 - o *GiraDestra/Sinistra* cambiano l'orientamento dell'agente di 90° .
- **Stati obiettivo**: quelli in cui tutte le celle sono pulite.
- **Costo di azione**: ogni azione ha un costo unitario.

b) Il rompicapo degli 8 tasselli:

È un gioco che consiste in una griglia 3x3, contenente 8 tasselli numerati da 1 a 8 e uno spazio vuoto. I tasselli possono essere spostati solo nell'unico spazio vuoto, facendo scorrere un tassello adiacente (sopra, sotto, destra o sinistra) nello spazio.

L'obiettivo è raggiungere una configurazione obiettivo, partendo da una configurazione iniziale mescolata.

Il problema degli 8 tasselli è studiato nell'ambito dell'intelligenza artificiale e della ricerca euristica perché:

- Ha un numero finito di stati (solo $9! = 362.880$ possibili permutazioni, ma solo metà sono risolvibili).
- Si presta bene a tecniche di ricerca del cammino ottimale, come: Ricerca in ampiezza (*breadth-first search*), A^* o Ricerca greedy.

3.3 - Algoritmi di ricerca

- Gli algoritmi di ricerca esplorano lo **spazio degli stati** per trovare soluzioni.
- Un aspetto cruciale è **gestire cammini ridondanti** (stati ripetuti, cicli) per evitare esplorazioni inutili o infinite.
- Gli algoritmi si valutano in termini di:
 - **Tempo** (quanti nodi vengono esplorati)
 - **Spazio** (quanta memoria è necessaria)

Molti algoritmi di ricerca sovrappongono allo spazio degli stati un albero di ricerca. Ciascun nodo corrisponde a uno stato nello spazio degli stati e i rami corrispondono ad azioni. Possiamo espandere ogni nodo, considerando le azioni disponibili per quello stato, vedendo dove queste portano e generando un nodo, detto figlio, per ognuno degli stati risultanti. I nodi non espansi che confinano con il resto dei nodi non ancora raggiunti, costituiscono la **frontiera**.

Ricerca Best-First (sistemare questo pezzetto):

Viene scelto un nodo n sulla frontiera corrispondente al valore minimo restituito da una funzione di valutazione $f(n)$.

Le prestazioni di un algoritmo di ricerca sono valutate secondo quattro criteri:

- **Completezza**: l'algoritmo deve trovare una soluzione se esiste, o riportare il fallimento.
- **Ottimalità rispetto al costo**: la soluzione deve avere il costo di cammino minimo.

- **Complessità temporale:** l'efficienza dell'algoritmo in termini di tempo.
- **Complessità spaziale:** la quantità di memoria utilizzata per effettuare la ricerca.

Non sempre lo spazio degli stati è finito o esplicito. In tal caso la complessità viene misurata in termini di **profondità**, ponendo un numero massimo di azioni per qualsiasi cammino e definendo un **fattore di ramificazione**, un numero che indica i successori di un nodo da considerare.

3.4 - Strategie di Ricerca Non Informata

Le strategie di ricerca non informata, chiamate anche "*cieche*", non usano alcuna conoscenza aggiuntiva sul problema, come ad esempio una stima della distanza dall'obiettivo. Si basano solo sulle informazioni contenute nella definizione iniziale del problema e si distinguono tra loro principalmente per l'ordine con cui espandono i nodi presenti nella frontiera di ricerca.

Ricerca in Ampiezza

La ricerca in ampiezza espande prima il nodo radice, poi tutti i suoi successori, quindi i successori di questi, procedendo livello per livello. Per gestire i nodi utilizza una coda di tipo FIFO, cioè i nodi inseriti per primi sono quelli che vengono espansi per primi.

È una strategia completa: se esiste una soluzione, verrà trovata. Inoltre, è ottimale se tutte le azioni hanno lo stesso costo. Tuttavia, ha una complessità temporale e spaziale elevata, perché il numero di nodi da esplorare cresce rapidamente con la profondità della soluzione.

Ricerca a Costo Uniforme (tra cui Algoritmo di Dijkstra)

Questa strategia estende la ricerca in ampiezza, ma invece di espandere i nodi in ordine di inserimento, sceglie sempre quello con il **costo cumulativo più basso** (ovvero il costo del cammino dal nodo iniziale). È quindi più adatta a problemi in cui le azioni possono avere costi diversi.

Come la ricerca in ampiezza, anche questa è **completa** e **ottimale**. Le sue prestazioni dipendono dalla struttura del problema e dalla distribuzione dei costi. *L'Algoritmo di Dijkstra* è una variante più specifica della ricerca a costo uniforme, dove non esiste un obiettivo da raggiungere e dove il calcolo continua fin quando tutti i nodi non vengono rimossi dalla coda con priorità.

Se chiamiamo con il termine **confine** l'insieme dei nodi ottimi ad ogni passo della ricerca, possiamo vedere che nella ricerca a costo uniforme le linee di confine corrispondono a dei cerchi concentrici a partire dalla radice.

Ricerca in Profondità

La ricerca in profondità si comporta in modo opposto rispetto a quella in ampiezza: espande sempre il nodo più profondo presente nella frontiera. Utilizza una struttura dati a pila, in cui l'ultimo nodo inserito è il primo a essere espanso.

Questa strategia **non è completa se esistono cammini infiniti**, poiché potrebbe perdersi esplorando un ramo all'infinito. Inoltre, **non è ottimale**, ovvero potrebbe trovare una soluzione meno conveniente rispetto ad altre. Il suo principale vantaggio è che richiede poca memoria, in quanto tiene traccia solo del percorso attuale.

Ricerca a Profondità Limitata e Approfondimento Iterativo

Ricerca a profondità limitata: è una variante della ricerca in profondità in cui si impone un limite massimo alla profondità esplorata. I nodi oltre questo limite non vengono espansi. Questa strategia non è completa se la soluzione si trova a una profondità maggiore del limite scelto, e non è ottimale se esiste una soluzione più vicina.

Ricerca ad approfondimento iterativo:

Combina i vantaggi della ricerca in ampiezza e in profondità. Consiste nell'eseguire ripetutamente una ricerca a profondità limitata, aumentando il limite a ogni iterazione. Parte con limite zero, poi uno, poi due e così via, fino a trovare una soluzione. Anche se può sembrare inefficiente, il costo aggiuntivo dovuto alla ripetizione dei livelli più superficiali è molto basso rispetto al numero di nodi da esplorare nei livelli più profondi.

Questa strategia è completa, ottimale nel caso di costi uniformi, e richiede poca memoria.

Ricerca Bidirezionale

La ricerca bidirezionale lavora su due fronti: una ricerca parte dallo stato iniziale e procede in avanti, mentre l'altra parte dallo stato obiettivo e procede all'indietro. L'algoritmo si ferma quando le due ricerche si incontrano.

Questa strategia può ridurre in modo significativo la complessità della ricerca, perché ciascuna delle due esplorazioni copre solo metà dello spazio. È utile soprattutto in problemi con un alto fattore di ramificazione. È completa e ottimale,

purché entrambe le ricerche utilizzino strategie corrette (come ampiezza o costo uniforme). Richiede però che sia possibile calcolare i predecessori degli stati, cosa non sempre fattibile.

3.5 - Strategie di ricerca informata (euristica)

Queste usano una **funzione euristica $h(n)$** per stimare il costo dal nodo corrente all'obiettivo. Ciò guida la ricerca in modo più efficiente. Esistono due strategie principali:

Ricerca best-first greedy:

Si tratta di una strategia di ricerca informata, la quale espande il nodo che **sembra più vicino all'obiettivo** secondo la funzione **$h(n)$** .

Mediante l'utilizzo di una **coda con priorità**, ordinando i nodi in base a $h(n)$, sceglie sempre il nodo più promettente sul momento.

L'efficienza dipende dalla **qualità dell'euristica**. Con una buona stima può essere molto più veloce delle strategie non informate, ma la complessità può comunque essere **esponenziale**.

Esempio:

Nella mappa della Romania, l'algoritmo può scegliere città vicine a Bucharest in linea d'aria, ma non necessariamente con il percorso più economico.

Limiti:

- **Non ottimale:** non garantisce il percorso a costo minimo.
- **Non completa:** può non trovare una soluzione anche se esiste.

Ricerca A*:

L'algoritmo A* è una delle strategie più importanti tra le ricerche informate, perché **combina due elementi fondamentali:**

- **$g(n)$:** il costo effettivo per arrivare a un nodo n partendo dallo stato iniziale;
- **$h(n)$:** una stima euristica del costo per arrivare dalla posizione attuale all'obiettivo.

Questi due valori vengono sommati in una funzione chiamata **$f(n) = g(n) + h(n)$** . L'algoritmo espande sempre il nodo con il valore di $f(n)$ più basso tra quelli presenti nella **frontiera**, che è gestita tramite una coda con priorità.

Questa strategia può essere vista come una **fusione tra la ricerca a costo uniforme** e la **ricerca best-first greedy**. A differenza della greedy, però, A* **considera anche il costo già accumulato**, non solo quello stimato.

Una proprietà chiave di A* è che, **se l'euristica è ammissibile** (se non sovrastima mai il costo reale), allora l'algoritmo è **completo** e **ottimale**.

Anche se nel **caso peggiore** la complessità può essere esponenziale, nella pratica A* può essere molto efficiente se si utilizza un'euristica ben progettata. Per questo motivo, è spesso la **scelta preferita** quando si vuole ottenere una soluzione ottimale in modo intelligente e guidato.

Esempio:

Nella mappa della Romania, A* può usare la **distanza in linea d'aria** come $h(n)$, mentre $g(n)$ rappresenta la somma dei costi reali delle strade percorse fin lì.

Diversamente dalla ricerca in ampiezza o in profondità, A* esegue la **potatura** dell'albero di ricerca, scartando i nodi non necessari a trovare una soluzione ottima, anche se questi sono figli di un nodo espanso. Questo principio rende l'algoritmo molto efficiente e rappresenta un pattern largamente utilizzato nel campo IA.

Le **linee di confine** di A*, invece di essere cerchi precisi concentrici, sono più simili a dei **cerchi tirati** o figure ovoidali espanse verso l'obiettivo.

Ricerca soddisfacente e ricerca A pesata:*

Quando si affrontano problemi di grandi dimensioni, trovare **una soluzione ottimale** può essere troppo costoso in termini di tempo e risorse. In questi casi può essere più utile cercare **una soluzione "abbastanza buona"** trovata velocemente. È proprio questo l'obiettivo delle strategie di **ricerca soddisfacente**.

Queste tecniche cercano un **compromesso tra qualità della soluzione e velocità di calcolo**, rinunciando all'ottimalità garantita (come quella di A*) in favore dell'efficienza. Due approcci principali vengono presentati:

1. **Ricerca informata basata su Euristiche Inammissibili**

Un'euristica è inammissibile quando **può sovrastimare** il costo reale verso l'obiettivo. Anche se questo toglie la garanzia di trovare il miglior percorso, consente all'algoritmo di **esplorare meno nodi** e quindi **trovare soluzioni più rapidamente**.

2. **A* pesata (Weighted A*)**

Si tratta di una variante di A* che **modifica la funzione di valutazione**: invece di usare $f(n) = g(n) + h(n)$, si usa $f(n) = g(n) + W \times h(n)$, dove **W è un peso maggiore di 1**. In questo modo, l'algoritmo **dà più importanza**

all'euristica, dirigendosi più velocemente verso l'obiettivo, anche se ciò può comportare l'espansione di cammini più costosi.
Questo metodo è **più rapido** e riduce il numero di nodi esaminati, ma **perde l'ottimalità**. Rimane comunque **completo**, a condizione che i costi siano positivi e il peso W sia finito.

Algoritmi A e ricerche best-first e ottimizzazioni della memoria:*

Gli algoritmi come **A*** e quelli **best-first** sono molto potenti, ma hanno un grande limite: devono **memorizzare tutti i nodi generati** finché non vengono espansi. Questo può causare problemi seri di **esaurimento della memoria**, soprattutto in spazi di ricerca molto vasti.

Per risolvere questa criticità, sono stati sviluppati algoritmi che **funzionano anche con una quantità limitata di memoria**. In particolare, vengono descritti due metodi efficaci:

1. **RBFS – Recursive Best-First Search**

RBFS è un algoritmo **ricorsivo** che simula il comportamento di **A***, ma **senza dover memorizzare tutti i nodi**.

Funziona così:

- Tiene traccia del miglior valore f alternativo disponibile dai nodi “antenati”.
- Se il valore f del nodo corrente supera questa soglia, l'algoritmo **torna indietro** (backtracking).
- Durante questo processo, il valore f viene “**riportato indietro**” al genitore, che così ha un'idea aggiornata della “promessa” di quella branca.
- In ogni momento, esplora il ramo che **sembra più promettente**, cioè con il valore f più basso.

Vantaggi:

Ha complessità spaziale lineare: usa molta meno memoria rispetto ad **A***.

- È **completo e ottimale** se i costi sono uniformi.

Limiti:

- Può ripetere il lavoro già fatto: tornando indietro e poi riesplorando, **genera più volte gli stessi nodi**, con un impatto sul tempo di esecuzione.

2. **SMA* – Simplified Memory-bounded A***

SMA* è una versione di A* pensata per funzionare in condizioni di **memoria limitata**.

Funziona così:

- Quando la memoria si riempie, **rimuove il nodo peggiore** (quello con valore *f*-costo più alto).
- Prima di rimuoverlo, **salva il suo valore *f*** nel nodo genitore, così non perde del tutto le informazioni su quella possibile strada.
- Questo meccanismo consente all'algoritmo di "ricordarsi" dei percorsi potati, se in futuro dovessero tornare utili.

Vantaggi:

- È **completo** se la memoria basta per contenere almeno il cammino alla soluzione più vicina.
- È **ottimale** se la memoria basta per memorizzare il cammino ottimale.

Limiti:

- Anche SMA* può dover **rigenerare nodi già esplorati**, aumentando i tempi di ricerca.
- Le prestazioni dipendono fortemente dalla **quantità di memoria disponibile**.

Ricerca bidirezionale euristica:

La ricerca bidirezionale consiste nel far partire due ricerche contemporaneamente: una dal **nodo iniziale verso l'obiettivo** e l'altra dall'**obiettivo verso il nodo iniziale**. L'algoritmo termina quando le due **frontiere** (ovvero gli insiemi di nodi ancora da esplorare) si incontrano. La soluzione si ottiene unendo i due percorsi che si incrociano nel nodo d'incontro.

Per rendere la strategia **informata**, si usano due funzioni di valutazione, una per ciascuna direzione:

- **Ricerca in avanti:** $f_F(n) = g_F(n) + h_F(n)$
- **Ricerca all'indietro:** $f_B(n) = g_B(n) + h_B(n)$
dove *g* rappresenta il costo del cammino finora percorso e *h* è la stima del costo rimanente.

Vantaggi:

Il **principale vantaggio** è il guadagno esponenziale in termini di tempo. Invece di esplorare fino alla profondità *d* (come una ricerca A* classica, che visita circa $O(b^d)$

nodi), ciascuna ricerca si ferma a circa $d/2$. Questo riduce il numero totale di nodi a circa $O(b^{(d/2)} + b^{(d/2)})$, che è molto più efficiente.

Difficoltà:

L'implementazione però non è banale:

- È **difficile garantire l'ottimalità** della soluzione semplicemente trovando un nodo comune alle due frontiere.
- Una strategia efficace è quella di **scegliere il nodo da espandere** che **minimizza** il valore dato dall'espressione $\max(f_F(n), f_B(n))$ o $f_F(n) + f_B(n)$, per far incontrare le due ricerche in un punto bilanciato.
- Bisogna anche **controllare costantemente se un nodo espanso da una direzione esiste già nella frontiera dell'altra**, per tenere traccia del miglior percorso possibile.

3.5 - Funzioni euristiche

Un'euristica, indicata con $h(n)$, serve a stimare quanto ci costerà raggiungere l'obiettivo a partire da un certo nodo n . Negli algoritmi di ricerca informata come A^* , la qualità dell'euristica può fare una grande differenza in termini di efficienza.

Per garantire che l'algoritmo A^* trovi sempre una soluzione ottimale, l'euristica deve essere **ammissibile**, cioè non deve mai sovrastimare il costo reale per arrivare alla meta. Ancora meglio se è anche **consistente** (o monotona): in questo caso, la funzione $f(n) = g(n) + h(n)$ non diminuisce mai lungo il cammino, rendendo la ricerca più ordinata e prevedibile.

Più un'euristica è **accurata** ($h(n) \approx h^*(n)$), meno nodi devono essere esplorati. In generale maggiore è la precisione di un'euristica, più piccolo è lo spazio degli stati esplorabile, dunque la ricerca è più efficiente.

3.6 - Buone euristiche: problemi rilassati e basi di pattern

Ma come si crea un'euristica efficace? Una strategia molto utile consiste nel partire da un **problema rilassato**, cioè una versione semplificata del problema originale ottenuta togliendo alcuni vincoli. L'idea è che, se il problema è più facile, possiamo risolverlo più velocemente e usare il costo di quella soluzione come una **stima al ribasso** per il problema originale. E questa stima sarà sempre **ammissibile**.

Prendiamo di nuovo l'8-puzzle come esempio:

- Per ottenere $h_{\square}$, si immagina un puzzle in cui i tasselli possono andare in qualsiasi posizione, anche non adiacente: l'unico criterio è se sono al posto giusto o no.

- Per $h \square$, si rilassa il vincolo che richiede uno spazio vuoto per muovere un tassello, e si permette ogni mossa ortogonale. Così si calcola quante mosse servirebbero **idealmente** per portare ogni tassello al suo posto.

Un altro metodo molto potente per costruire euristiche è quello delle **pattern databases (basi di pattern)**. Qui si prende un **sottoproblema** (per esempio, solo i tasselli 1-4 e lo spazio vuoto), lo si risolve completamente, e si memorizzano in una tabella tutti i costi delle soluzioni ottimali per ogni configurazione possibile di quel sottoinsieme. Per i tasselli 1-4, ad esempio, ci sono 15.120 configurazioni diverse che vengono pre-analizzate.

Il costo registrato in questa tabella può essere usato come euristica ammissibile nel problema più grande. E si può fare ancora di più: se si costruiscono **più basi disgiunte**, ciascuna che riguarda un sottoinsieme diverso dei tasselli (per esempio $\{1,2,3,4\}$ e $\{5,6,7,8\}$), si possono **sommarle** per ottenere un'euristica ancora più precisa, ma sempre ammissibile.

Cap. 5 – Agenti in ambienti competitivi

Il Capitolo 5 affronta un tipo particolare di ambienti: quelli in cui gli agenti non sono soli a pianificare, ma si trovano **in competizione** con altri agenti che hanno **obiettivi contrastanti**. Si parla quindi di **ricerca con avversari**, dove la scelta delle azioni non dipende solo dall'ambiente, ma anche dalle possibili mosse degli altri.

5.1 – Teoria dei giochi: come ragionano gli agenti in conflitto

La prima sezione introduce i concetti base della **teoria dei giochi**, applicati a situazioni in cui due o più agenti devono agire in uno stesso ambiente ma con obiettivi opposti. Il focus è sui cosiddetti **giochi a somma zero** tra **due giocatori**, dove il guadagno di uno corrisponde esattamente alla perdita dell'altro. L'esempio classico? Il **tris (tic-tac-toe)**.

In questi giochi si assume che entrambi i giocatori abbiano **informazione perfetta**, cioè vedono l'intero stato del gioco in ogni momento. Non c'è nulla di nascosto: tutto è trasparente, come sulla scacchiera.

Ogni partita può essere rappresentata come un **albero di gioco**, dove:

- I **nodi** sono gli **stati** del gioco.
- Gli **archi** rappresentano le **mosse** (cioè le azioni eseguibili da un giocatore).
- Le funzioni principali sono:
 - **RISULTATO** (s, a): restituisce lo stato successivo dopo aver eseguito l'azione a nello stato s .

- UTILITÀ (s, p): definisce il **valore del risultato finale** per il giocatore p se la partita termina nello stato s .
- TERMINALE(s): verifica se lo stato s è un **nodo finale** del gioco.

Questa rappresentazione permette di analizzare razionalmente le decisioni che i giocatori dovrebbero prendere per **massimizzare le proprie probabilità di vittoria**.

5.2 – Prendere decisioni ottimali: l'algoritmo MINIMAX

Dato un albero di gioco, è possibile determinare la strategia ottima calcolando il valore **minimax** di ogni stato, ovvero l'utilità per MAX di trovarsi nello stato corrispondente:

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITÀ}(s, \text{MAX}) & \text{se } \text{È-TERMINALE}(s) \\ \text{MAX}_{a \in \text{AZIONI}(s)} \text{MINIMAX}(\text{RISULTATO}(s, a)) & \text{se } \text{DEVE-MUOVERE}(s) = \text{MAX} \\ \text{MIN}_{a \in \text{AZIONI}(s)} \text{MINIMAX}(\text{RISULTATO}(s, a)) & \text{se } \text{DEVE-MUOVERE}(s) = \text{MIN} \end{cases}$$

In uno stato non terminale, MAX preferirà muoversi verso uno stato di valore massimo, mentre MIN verso uno stato di valore minimo.

Una volta definito il modello, il passo successivo è capire **come un agente possa giocare al meglio**. La strategia fondamentale nei giochi a somma zero con due giocatori è il celebre **algoritmo MINIMAX**.

L'idea alla base di MINIMAX è semplice ma potente:

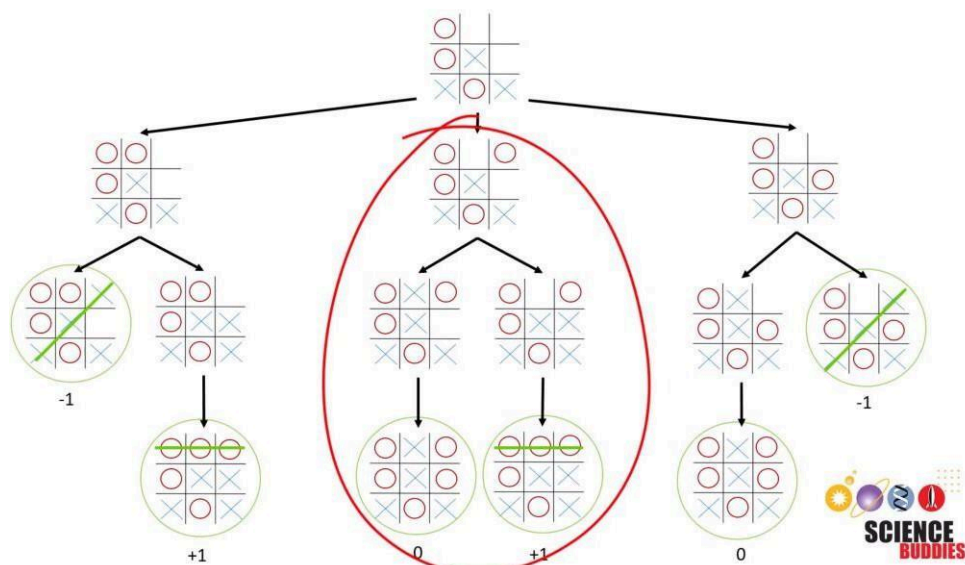
- Il giocatore MAX cerca sempre di **massimizzare** il proprio vantaggio.
- Ma deve tenere conto del fatto che l'altro giocatore, MIN, cercherà di **minimizzarlo** con le sue mosse.

Quindi MAX deve fare scelte **previdenti**, assumendo che l'avversario giochi in modo perfetto.

L'algoritmo funziona così:

1. Si parte dagli **stati terminali** dell'albero di gioco e si assegna a ciascuno un valore di **utilità**.
2. Questi valori vengono poi **propagati verso l'alto (backtracking)** nell'albero:
 - Se è il turno di MAX (livello dei nodi di MAX), si prende il **valore minimax massimo** tra quelli dei nodi figli, calcolato attraverso la funzione RISULTATO.
 - Se è il turno di MIN (livello dei nodi di MIN), si prende invece il **minimo**.

Bisogna ricordare che MINIMAX massimizza le possibilità di vincere, cercando di minimizzare le possibilità che vinca l'avversario. Questo significa che nella scelta della mossa migliore, verranno espansi i rami con più valori utilità “vittoria” o “pareggio” al posto di quelli con più “pareggio” o “sconfitta”.



Il ramo sinistro porta ai risultati sconfitta o vittoria, mentre quello centrale pareggio o vittoria. Quello di destra invece sconfitta o pareggio. Tra i tre nodi che MAX può scegliere, verrà selezionato quello centrale, in quanto le vie garantite sono una vittoria o almeno un pareggio. (MAX parte dal nodo radice, il livello successivo di nodi è la mossa di MIN e così via)

In questo modo, a ogni nodo viene assegnato un valore che rappresenta l'esito “atteso” del gioco se entrambi i giocatori agiscono in modo razionale.

Più precisamente, l'algoritmo MINIMAX esegue **un'esplorazione completa in profondità** dell'albero di gioco. Se la profondità massima è m , e in ogni punto ci sono b mosse legali, la complessità temporale dell'algoritmo sarà $O(b^m)$.

MINIMAX non è efficiente per giochi complessi. Per gli scacchi, con un fattore di ramificazione di 35 e una profondità media per match di 80 strati, dovrebbe effettuare ricerche su 35^{80} stati.

Giochi multiplayer:

Nei giochi multiplayer si può estendere il concetto di MINIMAX utilizzando per ogni valore singolo dei nodi, un vettore di valori per ciascun giocatore (v_A , v_B , v_C). Per gli stati terminali, la funzione utilità restituirà a sua volta un vettore. Per gli stati non terminali invece, verrà “portato su” il vettore contenente l'utilità più alta in uno dei suoi valori.

Minimax con potatura alfa-beta:

La **potatura alfa-beta** (*alpha-beta pruning*) rappresenta una strategia di ottimizzazione che permette di ottenere esattamente lo stesso risultato del MINIMAX

completo – cioè la stessa decisione ottimale – ma analizzando un numero molto più ridotto di nodi dell'albero. Il principio di base è semplice ma potente: **evitare di esplorare quelle parti dell'albero che non influenzeranno in alcun modo la decisione finale.**

Per fare ciò, l'algoritmo mantiene durante l'esplorazione ricorsiva dell'albero due valori importanti:

- **Alpha (α):** il miglior valore (cioè la massima utilità) che il giocatore MAX può ottenere lungo il percorso esplorato finora. In altre parole, rappresenta il minimo risultato che MAX si aspetta di ottenere se continua a giocare bene.
- **Beta (β):** il miglior valore (cioè il minimo risultato per MAX) che MIN può ottenere lungo il percorso esplorato. Indica il massimo che MIN può permettersi di concedere a MAX.

La **potatura vera e propria** avviene quando si capisce che proseguire l'esplorazione di un ramo dell'albero è inutile perché non potrà mai portare a una scelta migliore rispetto a quanto già trovato. Esistono due casi principali in cui ciò accade:

1. **Durante l'esplorazione di un nodo di MIN:** se si scopre che il valore minimo che MIN può garantire è già **inferiore o uguale ad α** , allora MAX ha già un'opzione migliore. Non c'è motivo di considerare ulteriori mosse in questo ramo, perché MAX non le sceglierà comunque. Quindi si può tagliare quel ramo (potare i figli del nodo).
2. **Durante l'esplorazione di un nodo di MAX:** se si scopre che il valore massimo che MAX può ottenere è già **maggiore o uguale a β** , allora MIN ha già un'alternativa migliore. Anche in questo caso, possiamo interrompere l'esplorazione del ramo.

In sostanza, **l'algoritmo evita di analizzare rami che sicuramente non verranno scelti da un avversario razionale**, risparmiando tempo e risorse.

Il vantaggio di questa tecnica è notevole: nei casi migliori, la complessità computazionale può scendere da $O(b^d)$ del MINIMAX standard a $O(b^{d/2})$, dove b è il fattore di ramificazione (cioè il numero medio di mosse possibili per ciascun stato), e d è la profondità dell'albero.

Infine, è importante sottolineare che **l'efficacia della potatura alfa-beta dipende molto dall'ordine in cui vengono esaminate le mosse**. Se l'algoritmo analizza prima le mosse migliori, riuscirà a potare molto di più. Al contrario, se le mosse meno promettenti vengono esplorate prima, si perderà gran parte del vantaggio.

Ordinamento delle mosse

Trovare in anticipo l'ordine perfetto delle mosse è, purtroppo, **impossibile senza risolvere completamente il problema**. Si può però migliorare l'ordinamento delle mosse usando diverse strategie:

- **Approfondimento iterativo**: una tecnica discussa nel Capitolo 3, in cui l'albero di gioco viene esplorato inizialmente a profondità basse e poi sempre più profonde. Le informazioni raccolte durante le ricerche superficiali (come quali mosse sembrano migliori) vengono poi **riutilizzate per ordinare le mosse nelle ricerche più profonde**, migliorando l'efficienza complessiva.
- **Euristiche killer**: questa tecnica dà **priorità alle mosse che in passato hanno causato potature significative (*cutoffs*)**, con l'idea che possano avere lo stesso effetto in rami simili dell'albero alla stessa profondità.

Tabelle di Trasposizione:

Uno dei problemi principali negli algoritmi di ricerca nei giochi, come **MINIMAX** o **alfa-beta**, è il fenomeno della **riesplorazione**: lo stesso stato può essere raggiunto attraverso sequenze diverse di mosse.

Per risolvere questo problema, si utilizzano le **tabelle di trasposizione**. Si tratta di una sorta di **memoria intelligente** che salva il valore di utilità calcolato per ogni stato visitato.

In pratica, quando l'algoritmo di ricerca si imbatte in una posizione, **prima di esplorarla controlla se quella posizione è già stata valutata**. Se la trova nella tabella, **recupera direttamente il valore memorizzato**, evitando così di riesplorare tutto il sottoalbero a partire da quello stato.

Questo approccio ha un effetto enorme sull'efficienza della ricerca, soprattutto nei giochi dove è **comune ritornare allo stesso stato da percorsi diversi**. In pratica, permette di trattare l'albero di gioco più come un **grafo**, dove i nodi non vengono duplicati inutilmente. Inoltre, **l'accesso alla tabella è molto veloce** (tempo costante nella maggior parte dei casi), rendendo questa ottimizzazione particolarmente conveniente.

5.3 – Ricerca alfa-beta euristica

Quando si applica la **ricerca alfa-beta** in un contesto **euristico**, il problema principale è che non è possibile esplorare l'intero albero del gioco fino agli stati terminali, a causa della **profondità troppo elevata** o del **numero enorme di mosse possibili**. Per questo motivo, si introduce il concetto di **funzione di valutazione**.

Funzioni di Valutazione

La funzione di valutazione (indicata come *FUNZIONE-VALUTAZIONE* o *EVAL*) permette all'algoritmo di **interrompere la ricerca a una certa profondità** e di **stimare il valore di uno stato non terminale**. In pratica, anziché arrivare fino alla fine della partita, si fa una “fotografia” del gioco in un certo momento e si prova a capire, in base a quella, **quanto sia promettente la posizione per il giocatore MAX**.

$$H\text{-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{se TEST-TAGLIO}(s, d) \\ \text{MAX}_{a \in \text{AZIONI}(s)} H\text{-MINIMAX}(\text{RISULTATO}(s, a), d+1) & \text{se DEVE-MUOVERE}(s) = \text{MAX} \\ \text{MIN}_{a \in \text{AZIONI}(s)} H\text{-MINIMAX}(\text{RISULTATO}(s, a), d+1) & \text{se DEVE-MUOVERE}(s) = \text{MIN}. \end{cases}$$

Esempio:

In una partita a scacchi, invece di arrivare al termine della partita per sapere se si vince o si perde, ci si ferma dopo alcune mosse e si valuta la posizione attuale in base ai pezzi presenti, al controllo del centro, alla sicurezza del re, ecc.

Differenza tra funzione di utilità e funzione di valutazione:

- La **funzione di utilità** è **esatta** e si usa **solo negli stati terminali**: assegna un valore come +1 (vittoria di MAX), 0 (pareggio) o -1 (vittoria di MIN).
- La **funzione di valutazione (EVAL)**, invece, è **una stima euristica**, applicata agli **stati non terminali**, per valutare quanto un certo stato sia favorevole al giocatore MAX.

Una funzione di valutazione efficace deve:

- Essere **computazionalmente efficiente**, cioè veloce da calcolare.
- Essere **ben correlata** con la reale probabilità di vittoria: più alto è il valore della funzione, maggiori dovrebbero essere le probabilità che MAX vinca da quella posizione.

Implementazione:

Di solito, si identificano una serie di **caratteristiche (features)** dello stato di gioco e si assegnano loro dei **pesi**. La valutazione è spesso una **combinazione lineare** (tuttavia non sempre) di queste caratteristiche:

$$EVAL(s) = w_1 \cdot f_1(s) + w_2 \cdot f_2(s) + \dots + w_n \cdot f_n(s)$$

dove:

- $f_i(s)$ rappresenta una caratteristica dello stato (es. numero di torri rimaste),
- w_i è il peso assegnato a quella caratteristica.

Le caratteristiche dovrebbero essere **normalizzate** per evitare squilibri tra i diversi termini. Ad esempio, si potrebbe dividere ogni valore per il massimo valore possibile, ottenendo così numeri tra 0 e 1.

Esempio pratico: gli scacchi

Nel gioco degli scacchi, una funzione di valutazione semplice ma sorprendentemente efficace consiste nel **valutare il materiale** rimasto:

- Pedone = 1 punto
- Cavallo e Alfiere = 3 punti
- Torre = 5 punti
- Regina = 9 punti

Sommando questi valori per ciascun giocatore e calcolando la differenza, si ottiene una stima utile dello stato di gioco. Funzioni più avanzate includono anche:

- Controllo del centro
- Sicurezza del re
- Mobilità delle pedine
- Posizioni dominanti

Test-Taglio dell'albero di ricerca

Nel contesto della **ricerca alfa-beta euristica**, una questione fondamentale è **stabilire fino a che punto esplorare l'albero di gioco**, dunque è necessario introdurre un **meccanismo per interrompere la ricerca** a una certa profondità e **valutare gli stati raggiunti in modo euristico**.

Il processo di “*taglio*” si basa sull’uso della funzione **TEST-TAGLIO**(**stato**, **profondità**), la quale **decide se la ricerca debba proseguire o se ci si debba fermare** e applicare la funzione di valutazione EVAL. Tipicamente, questo avviene quando viene raggiunto un **limite massimo di profondità**.

Esempio:

In uno scenario dove si imposta un limite di profondità pari a 4, l’algoritmo esplorerà solo 4 livelli dell’albero. Alla profondità 4, EVAL verrà chiamata per stimare il valore dello stato.

Questo approccio è una **variazione della ricerca ricorsiva**, ma spesso si integra con una tecnica chiamata **approfondimento iterativo**, che consiste nell’eseguire più ricerche successive aumentando gradualmente la profondità massima.

Stati quiescenti:

Il semplice taglio della ricerca a una profondità fissa può portare a **valutazioni sbagliate**. Questo perché **l’euristica può essere ingannata** da situazioni instabili: uno stato potrebbe sembrare vantaggioso, ma solo perché non si è vista una minaccia imminente (come una perdita forzata o uno scacco).

Per affrontare questo problema si introduce il concetto di **stati quiescenti**. Uno stato è quiescente se **non ci sono minacce o opportunità tattiche immediate**, come:

- Prese (catture di pezzi importanti)
- Scacchi
- Mosse che cambiano drasticamente la situazione sulla scacchiera

L’idea è che **la funzione di valutazione venga applicata solo in posizioni “tranquille”**, dove l’euristica è più affidabile. Se lo stato raggiunto **non è quiescente**, l’algoritmo può continuare la ricerca localmente fino a trovare una posizione più stabile.

Questo approccio, chiamato **ricerca quiescente (quiescence search)**, riduce il rischio di prendere decisioni basate su valutazioni “truffaldine” di stati troppo dinamici.

Effetto orizzonte:

L’effetto orizzonte è un **problema nella ricerca euristica a profondità limitata**, in cui l’algoritmo **non riesce a “vedere oltre” un certo numero di mosse**.

Un evento negativo inevitabile (come la perdita di un pezzo importante) potrebbe avvenire **poco dopo** il limite di profondità. L'avversario può quindi **usare mosse dilatorie** (come scacchi o sacrifici) per **ritardare** l'evento oltre il "confine" della ricerca.

Il programma valuta la posizione senza rendersi conto del pericolo imminente e **stima erroneamente la situazione come favorevole**, solo perché la minaccia è nascosta oltre il suo "orizzonte" di ricerca.

Per mitigare il problema, si possono usare **estensioni semplici** della ricerca, ovvero continuare ad analizzare alcune mosse "*importanti*" (come scacchi o catture) **anche oltre il limite di profondità**, per **vedere oltre l'orizzonte** ed evitare valutazioni errate.

Potatura in avanti (forward pruning):

È una **tecnica euristica** usata per ridurre il numero di mosse esplorate durante la ricerca. L'idea è di scartare **in anticipo** i rami che sembrano deboli secondo una valutazione euristica, **senza calcolarne il valore esatto**.

Diversamente dalla potatura alfa-beta, **non garantisce** di trovare la mossa ottimale. Qui sotto sono presenti alcune varianti di forward pruning:

Beam Search:

Un approccio di potatura in avanti è la **Beam Search**, per la quale si esplorano solo le **N mosse migliori** per ogni nodo, secondo la funzione EVAL; le altre si ignorano.

Tuttavia, questa tecnica ha i suoi difetti, difatti una mossa scartata potrebbe rivelarsi vincente più avanti. Inoltre, per effetto orizzonte vi è il rischio di tagliare una linea che nasconde minacce a lungo termine.

ProbCut:

Il **ProbCut** è una tecnica probabilistica, adottata da programmi come *Othello*, *Logistello* e *Trov*. Funziona eseguendo una valutazione dello stato s a una certa profondità d , ottenendo un valore $V(s,d)$. Sulla base di questo valore, il programma **stima la probabilità** che il vero valore minimax del nodo sia superiore o inferiore ai limiti alfa o beta.

Se questa probabilità è sufficientemente alta (per esempio, superiore al 64%), il ramo viene potato perché è **molto probabile** che non influenzi il risultato finale.

In pratica, **si evita di esplorare rami considerati poco promettenti**, secondo un'analisi statistica dei risultati ottenuti da valutazioni a profondità inferiori.

Riduzione delle ultime mosse (Late Move Reduction):

Questa tecnica si basa su un'osservazione empirica: le mosse migliori tendono a essere tra le **prime considerate** da un'euristica di ordinamento.

Al contrario, le mosse che vengono esaminate più tardi sono generalmente meno promettenti. L'idea, quindi, è di **ridurre la profondità di ricerca** per queste mosse "tardive", esplorandole meno a fondo rispetto alle prime. Questo consente di risparmiare tempo su mosse che, con alta probabilità, **non porteranno a un miglioramento significativo** della valutazione della posizione.

Ricerca su albero e ricerca su tabelle:

Gli algoritmi come Alpha-Beta usano tabelle per **memorizzare le valutazioni di posizioni già esplorate**, evitando di riesaminare gli stessi stati più volte. Questo **velocizza notevolmente la ricerca**. Durante l'apertura e la chiusura della partita, le mosse possono essere **recuperate** da tabelle invece che calcolate.

Per costruire **tabelle complete dei finali di partita**, una **ricerca minimax retrograda** parte da posizioni terminali (es. scaccomatto) e risale determinando i valori delle posizioni precedenti. Se questi corrispondono a "vittoria", la posizione viene salvata nella tabella.

In seguito, se durante la ricerca alfa-beta si incontra una posizione presente nelle **tabelle dei finali**, il programma può usare **il valore esatto** già noto, migliorando la precisione della valutazione.

Cap. 6 – Problemi di soddisfacimento di vincoli

Nei sistemi di ricerca euristica gli stati sono *atomici* (indivisibili), mancano le informazioni di dominio necessarie per descrivere le transizioni tra gli stati.

Si può ovviare a ciò, attraverso una **rappresentazione fattorizzata** per ogni stato: un insieme di **variabili** aventi ciascuna un **valore**. Quando ciascuna variabile ha un valore in grado di soddisfare i vincoli, il problema può dirsi risolto.

Questo pattern è detto **problema di soddisfacimento di vincoli**, o **CSP** (*constraint satisfaction problem*). L'idea è quella di eliminare ampie porzioni dello spazio di ricerca tutte insieme, individuando le variabili che violano i vincoli.

6.1 - Definizione dei problemi di soddisfacimento di vincoli o CSP:

Un problema di soddisfacimento dei vincoli è costituito da tre componenti X, D, C :

- X è un insieme di **variabili** $\{X_1, \dots, X_n\}$
- D è un insieme di **domini** $\{D_1, \dots, D_n\}$ per ciascuna variabile
- C è un insieme di **vincoli** che specificano combinazioni di valori ammesse

Ciascun dominio D_i è costituito da un insieme di valori ammessi $\{v_1, \dots, v_n\}$ per la variabile X_i .

Ogni vincolo C_i , è costituito da una coppia $\square \text{ambito}, \text{rel} \square$, dove **ambito** rappresenta le variabili che partecipano al vincolo, mentre **rel** è una relazione che definisce i valori assumibili da tali variabili (*può essere sia espressa in modo esplicito, sia come funzione*).

Esempio:

Se X_1 e X_2 hanno dominio $\{1, 2, 3\}$, il vincolo che afferma che X_1 deve essere maggiore di X_2 si può scrivere come $\square (X_1, X_2), \{(3,1), (3,2), (2,1)\} \square$ o come $\square (X_1, X_2), X_1 > X_2 \square$.

Nei CSP si effettuano **assegnamenti** di valori alle variabili. Un assegnamento che **non viola alcun vincolo** è detto **consistente** (o legale). Inoltre, un assegnamento è **completo** se a tutte le variabili è **assegnato** un valore. Una **soluzione** di un problema CSP è un assegnamento completo e consistente. Una soluzione **parziale** è un assegnamento parziale (*non tutte le variabili sono assegnate*) e consistente.

Problemi esempio:

Colorazione di una mappa:

Guardare p. 186 IV edizione

Si vuole colorare gli stati australiani in una cartina in modo tale che le regioni adiacenti abbiano colori distinti di dominio {rosso, verde, blu}.

Il CSP si può visualizzare come un **grafo di vincoli**. Ciascun nodo corrisponde a una variabile, e gli archi che si formano tra due variabili indicano la partecipazione a un vincolo.

I CSP costituiscono una rappresentazione naturale per un'ampia varietà di problemi e sono in grado di **potare rapidamente grandi porzioni dello spazio degli stati**, dunque sono molto efficienti. Difatti, non appena un assegnamento parziale viola un vincolo, si può immediatamente scartare ulteriori raffinamenti di quell'assegnamento.

Programmazione dei lavori:

Guardare p. 187 IV edizione

Varianti del formalismo CSP:

Natura dei domini:

Nei problemi a vincoli (CSP), le variabili possono avere domini di diversa natura:

- **Domini discreti finiti**
Sono i più frequenti nei CSP. In questi casi, ogni variabile può assumere solo un numero **finito** di valori. Esempi classici includono problemi come la colorazione di mappe, il problema delle otto regine o il Sudoku. In tutti questi casi, si lavora con un insieme ristretto di possibili scelte.
- **Domini discreti infiniti**
Qui le variabili possono assumere un numero potenzialmente **illimitato** di valori interi. Un esempio tipico è la programmazione dei lavori (job scheduling), dove le variabili possono rappresentare istanti temporali non limitati. In questi contesti, i vincoli sono spesso di tipo lineare.
- **Domini continui**
Si tratta di problemi in cui le variabili possono assumere valori **reali** all'interno di un intervallo. Sono tipici della programmazione matematica e dell'ottimizzazione continua, come nella programmazione lineare o quadratica. Questi problemi appartengono più propriamente al campo della matematica applicata.

Natura dei vincoli:

I vincoli specificano quali combinazioni di valori sono ammissibili. A seconda di quante variabili coinvolgono, possiamo distinguere:

- **Vincoli unari**
Agiscono su una **singola variabile**. Sono semplici da verificare e spesso servono a **restringere il dominio**. Un esempio può essere un vincolo che impone a una regione su una mappa di essere colorata di verde.
- **Vincoli binari**
Coinvolgono **due variabili** alla volta. Se tutti i vincoli di un CSP sono **binari**, possiamo rappresentare il problema come un **grafo**, dove i nodi sono le variabili e gli archi rappresentano i vincoli. Esempi classici includono la colorazione di mappe (dove regioni adiacenti devono avere colori diversi) o il problema delle otto regine, in cui ogni coppia di regine deve rispettare certi vincoli di posizione.
- **Vincoli globali**
Coinvolgono un **numero arbitrario di variabili** e, anche se in teoria possono

essere scomposti in vincoli binari, è spesso più efficiente trattarli con algoritmi dedicati. Un vincolo molto noto è il vincolo **Alldiff** (tutti diversi), usato ad esempio nel Sudoku o nei problemi di critto-aritmetica, dove ogni cifra o simbolo deve essere unico. Un altro esempio è il vincolo **Atmost**, che impone che un certo valore possa essere assegnato solo a un numero limitato di variabili.

Vincoli di preferenza:

In alcuni problemi non tutti i vincoli devono essere rispettati in modo rigido. Esistono vincoli cosiddetti “*morbidi*” o **di preferenza**, che rappresentano desideri piuttosto che obblighi. In questi casi, l’obiettivo non è solo trovare una soluzione che rispetti i vincoli obbligatori, ma anche una soluzione che soddisfi il maggior numero possibile di preferenze. Questo tipo di problemi prende il nome di **Constraint Optimization Problems (COP)**. Un esempio tipico è la programmazione degli orari scolastici o universitari, dove possono esserci preferenze sugli orari delle lezioni, sulle aule disponibili o sui docenti assegnati.

6.2 – Propagazione dei vincoli: inferenza nei CSP:

Questo capitolo esplora come l’inferenza possa essere impiegata per rendere più efficiente la risoluzione dei **CSP**. In particolare, si mostra come sia possibile **ridurre lo spazio di ricerca** già prima — o durante — l’applicazione degli algoritmi di ricerca, sfruttando la struttura del problema stesso.

L’idea centrale è semplice ma potente: se possiamo **escludere in anticipo** determinati valori dai domini delle variabili perché sappiamo già che non possono far parte di una soluzione valida, possiamo risparmiare moltissimo lavoro computazionale. Per farlo, si introducono i concetti di **consistenza locale**, che aiutano a verificare la coerenza di porzioni limitate del problema.

Consistenza di nodo:

La consistenza di nodo è il livello più semplice di consistenza locale nei CSP.

Una variabile si dice **nodo-consistente** quando tutti i valori nel suo dominio rispettano i **vincoli unari**, cioè quelli che riguardano solo quella variabile. Rendere un CSP nodo-consistente significa esaminare ogni variabile e rimuovere dal suo dominio i valori che violano i vincoli unari associati. Se durante questo processo il dominio di una variabile si svuota, il problema non ha soluzioni.

Un esempio è quello della *colorazione della mappa australiana*: se alla regione WA fosse imposto il vincolo di essere verde, allora i colori rosso e blu verrebbero eliminati dal suo dominio per garantire la consistenza di nodo.

Consistenza d'arco:

La consistenza di arco è una **tecnica di inferenza** usata per semplificare i CSP, e riguarda le **relazioni binarie** tra coppie di variabili. Ogni variabile in un CSP ha un **dominio**, ovvero l'insieme dei valori che può assumere. Una variabile X_i è considerata **arco-consistente** rispetto a un'altra variabile X_j se, per ogni valore v nel dominio di X_i , esiste almeno un valore w nel dominio di X_j tale che la coppia (v, w) soddisfi il **vincolo binario** R_{ij} che lega le due variabili.

Applicare la consistenza di arco significa quindi **ripulire i domini delle variabili**, rimuovendo quei valori che non possono essere estesi a soluzioni valide quando si considerano i vincoli con le variabili vicine. Questo processo viene ripetuto finché non si possono più eliminare valori da alcun dominio.

Un algoritmo molto usato per rendere un CSP arco-consistente è **AC-3**, che lavora gestendo una coda di archi da controllare. Ogni volta che un arco viene analizzato e causa la rimozione di valori dal dominio di una variabile, gli archi che la coinvolgono vengono reinseriti in coda, poiché questa modifica potrebbe invalidare la consistenza con altre variabili.

L'algoritmo AC-3 opera su CSP binari e ha una complessità media di $O(n^2d^3)$, dove n è il numero di variabili e " d " la dimensione massima dei domini.

Consistenza di cammino:

La consistenza di cammino è un livello di inferenza più avanzato rispetto alla consistenza di nodo e a quella di arco, perché coinvolge **tre variabili** alla volta: una coppia e una variabile intermedia. Una coppia di variabili (X_i, X_j) è detta cammino-consistente rispetto a una terza variabile X_m se, per ogni assegnamento $\{X_i = a, X_j = b\}$ consistente con i vincoli, esiste un assegnamento per X_m che soddisfa i vincoli su $\{X_i, X_m\}$ e $\{X_m, X_j\}$.

Questo meccanismo consente di **raffinare i domini delle variabili** eliminando combinazioni di valori che non possono estendersi a una soluzione globale consistente. Per esempio, nel problema della colorazione della mappa australiana, dopo aver applicato la consistenza di arco tra le regioni WA e SA, è ancora possibile trovare combinazioni come (rosso, rosso). Tuttavia, considerando la variabile NT come intermedia, si scopre che non esiste alcun valore nel suo dominio che sia compatibile con entrambi (WA = rosso, SA = rosso), e quindi questa combinazione può essere scartata. In questo modo, la consistenza di cammino permette di ridurre i domini delle variabili in modo più mirato, migliorando l'efficienza della ricerca di una soluzione.

K-consistenza:

La k-consistenza è una generalizzazione delle tecniche di propagazione di vincoli precedenti:

- La **consistenza di nodo** corrisponde alla **1-consistenza**. Un problema è 1-consistente se il dominio di ogni variabile non è vuoto.
- La **consistenza di arco** equivale alla **2-consistenza**. Un CSP è 2-consistente se, per ogni coppia di variabili con un vincolo, ogni valore nel dominio di una variabile ha almeno un valore corrispondente nel dominio dell'altra che soddisfa il vincolo.
- La **consistenza di cammino** può essere vista come una forma di **3-consistenza**. Essa verifica che per ogni coppia di valori (v, w) di due variabili X_i e X_j che soddisfano il vincolo R_{ij} , esista un valore per una terza variabile X_m tale che siano soddisfatti anche i vincoli R_{im} e R_{mj} .

Più in generale, un CSP è detto **k-consistente** se, per ogni insieme di $k-1$ variabili e ogni assegnamento consistente a tali variabili, esiste sempre un valore nel dominio di una qualunque k-esima variabile che possa essere aggiunto all'assegnamento senza violare i vincoli. In altre parole, ogni assegnamento parziale consistente su $k-1$ variabili può essere **esteso** a una variabile aggiuntiva, mantenendo la consistenza.

Viene inoltre definita la **forte k-consistenza**: un CSP è fortemente k-consistente se è j-consistente per ogni $j \leq k$. Ad esempio:

- La **forte 2-consistenza** corrisponde alla consistenza di arco.
- La **forte 3-consistenza** richiede l'applicazione congiunta della consistenza di nodo, di arco e di cammino.

Rendere un CSP **fortemente k-consistente** per valori di $k > 2$ implica considerare vincoli impliciti che coinvolgono più di due variabili. Tuttavia, il costo computazionale cresce rapidamente con k , con complessità dell'ordine di $O(n^k d^k)$, dove n è il numero di variabili e d la dimensione massima dei domini. Per questo motivo, nella pratica si applicano raramente tecniche di forte k-consistenza per valori alti di k , limitandosi tipicamente a $k = 2$ o 3 , in quanto il beneficio in termini di riduzione della ricerca non compensa quasi mai il costo computazionale aggiuntivo.

Vincoli globali:

A differenza dei vincoli **unari** (che coinvolgono una sola variabile) e **binari** (che coinvolgono coppie di variabili), i **vincoli globali** permettono di esprimere in modo compatto relazioni complesse tra un numero arbitrario di variabili. Grazie alla loro struttura ricorrente, questi vincoli possono essere gestiti tramite **algoritmi**

specializzati che ne sfruttano le proprietà per ottenere una propagazione più efficace.

Un vincolo globale spesso ricorrente è **TutteDiverse**, il quale impone che tutte le variabili coinvolte assumano **valori distinti**. Una rappresentazione binaria di TutteDiverse, in cui ogni coppia di variabili è soggetta a un vincolo di disuguaglianza, non è in grado di catturare tutte le inconsistenze, pertanto l'algoritmo AC-3 risulta inefficiente. Al suo posto vengono utilizzati **algoritmi dedicati** che ricercano una forma di **consistenza di arco sull'intero vincolo globale**, piuttosto che sulle sole coppie di variabili. Quello più utilizzato ha una complessità dell'ordine di $O(k^2d^2)$, più efficiente di AC-3 con $O(n^2d^3)$. Altri vincoli spesso ricorrenti sono *AlPiù*, *AlMeno* e *Esattamente*.

Esempio:

Nel **problema delle 8 regine**, si può utilizzare un vincolo *AlPiù* per garantire che **al massimo una regina** si trovi su ciascuna diagonale della scacchiera.

Un altro vincolo globale importante è il vincolo **disgiuntivo**, usato nei problemi di **schedulazione (job scheduling)**, dove si impone che due attività non si sovrappongano nel tempo.

In sintesi, i vincoli globali offrono una **modellazione più espressiva e compatta** per i CSP e, se trattati con algoritmi di propagazione specifici, permettono di **aumentare la potenza inferenziale e l'efficienza** rispetto all'uso di tecniche generiche applicate a vincoli binari equivalenti.

Sudoku:

Il **Sudoku** è un puzzle logico che si basa su una griglia di **9×9 caselle**, alcune delle quali contengono cifre iniziali. L'obiettivo è riempire le caselle vuote con cifre da 1 a 9 in modo tale che:

- Ogni **riga** contenga tutte le cifre da 1 a 9 senza ripetizioni;
- Ogni **colonna** contenga tutte le cifre da 1 a 9 senza ripetizioni;
- Ciascuno dei **nove blocchi 3×3** (detti anche "unità") contenga tutte le cifre da 1 a 9 senza ripetizioni.

In termini di CSP, il Sudoku può essere modellato come un problema con **81 variabili**, una per ciascuna casella della griglia. Il **dominio** di ogni variabile è l'insieme $\{1, \dots, 9\}$, a meno che la casella non sia già precompilata con una cifra, nel qual caso il dominio è ridotto a un singolo valore.

I vincoli del Sudoku si basano sul vincolo globale **TutteDiverse**. In particolare, vi sono:

- 9 vincoli TutteDiverse, uno per ciascuna **riga**;
- 9 vincoli TutteDiverse, uno per ciascuna **colonna**;
- 9 vincoli TutteDiverse, uno per ciascun **blocco 3×3**.

In totale, il problema prevede **27 vincoli globali** di tipo TutteDiverse.

Anche se il Sudoku può essere risolto tramite **ricerca esaustiva** o con l'applicazione della **consistenza di arco** (ad esempio, mediante AC-3), l'efficacia di queste tecniche generiche è limitata. In particolare, la rappresentazione binaria del vincolo TutteDiverse non consente di rilevare tutte le inconsistenze. Per questo motivo, si preferisce utilizzare **algoritmi specializzati** per la propagazione del vincolo TutteDiverse, capaci di ottenere una consistenza più forte in modo più efficiente.

Nel contesto del Sudoku, è stato anche osservato che alcune **regole di inferenza specifiche**, basate su **pattern strutturali**, permettono di eliminare valori dai domini in modo efficace. Un esempio emblematico è quello delle **naked triplets** (*triplette nude*): un pattern in cui tre caselle all'interno di un'unità condividono esattamente tre possibili valori tra loro, permettendo di rimuovere tali valori dai domini delle altre caselle dell'unità.

Nonostante il Sudoku sia un problema **NP-completo**, nella pratica **molte istanze** possono essere risolte in modo efficiente **senza necessità di ricerca esplicita**, semplicemente applicando in modo iterativo **regole di inferenza logica**. Questo aspetto rappresenta uno degli elementi più affascinanti del Sudoku: per il risolutore umano, il processo è spesso percepito come un esercizio logico più che computazionale, basato sull'individuazione e l'applicazione di schemi e strategie inferenziali.

6.3 - Ricerca con backtracking per CSP:

L'algoritmo di **backtracking** seleziona una **variabile non assegnata**, prova a darle un **valore dal suo dominio** e continua ricorsivamente. Se l'assegnazione viola un vincolo, il valore viene scartato e si prova il successivo. Se nessun valore è valido, l'algoritmo torna indietro alla variabile precedente per cambiare scelta.

Ordinamento di variabili e valori:

La scelta dell'ordine con cui assegnare variabili e valori influisce molto sull'efficienza della ricerca.

- **Ordinamento statico delle variabili**

Un approccio semplice è quello statico, nel quale le variabili sono scelte in ordine fisso $\{X_1, X_2, \dots\}$.

L'**euristica MRV (Minimum Remaining Values)** seleziona la variabile con il

minor numero di valori legali rimasti. Seguendo il principio "*fail-first*", si tenta prima di risolvere le parti più vincolate, aumentando le probabilità di scoprire rapidamente conflitti.

- **L'euristica di grado (*degree heuristic*)** cerca di ridurre il fattore di ramificazione delle scelte future scegliendo la variabile coinvolta nel maggior numero di vincoli con le altre variabili assegnate. Torna utile nel caso si verifichino dei *pareggi* nell'ordinamento.
 - **Ordinamento dei valori**
Una volta scelta la variabile, conviene provare prima il **valore meno vincolante**, cioè quello che **riduce meno** i domini delle variabili collegate. Questo non riduce lo spazio di ricerca, ma può **portare più velocemente a una soluzione**, riducendo le volte in cui il backtracking sarà necessario.
-

Alternanza di ricerca e inferenza:

Quando si assegna un valore a una variabile, si può applicare **inferenza** per eliminare dai domini delle variabili non assegnate i valori che risultano subito inconsistenti. Questo aiuta a rilevare i fallimenti prima e a ridurre lo spazio di ricerca.

- **Verifica in avanti (*Forward Checking*)**

Dopo aver assegnato un valore a una variabile, si controllano le variabili vicine ancora non assegnate e si eliminano dai loro domini i valori incompatibili.

Se un dominio si svuota, si scarta subito l'assegnazione.

È una tecnica semplice che aiuta a potare l'albero di ricerca, mantenendo solo la **consistenza di arco locale**.

- **MAC (*Maintaining Arc Consistency*)**

Più potente della verifica in avanti, MAC mantiene la **consistenza di arco su tutto il CSP** dopo ogni assegnazione, usando ad esempio l'algoritmo AC-3. Se il dominio di una variabile si svuota, si torna indietro.

MAC è più efficace nel potare, ma più costoso in termini di calcolo.

Backtracking intelligente (*Backjumping*)

Il backtracking cronologico tradizionale torna sempre alla variabile precedentemente assegnata, anche se questa potrebbe non essere la causa del fallimento. Questo approccio può essere inefficiente.

Il **backjumping** migliora la ricerca identificando la vera **causa del conflitto**.

Quando una variabile X fallisce (nessun valore valido nel dominio), si costruisce un **insieme dei conflitti**: l'insieme delle variabili già assegnate che hanno causato l'inconsistenza.

L'algoritmo non torna indietro al genitore di X, ma **salta direttamente** all'ultima variabile nell'insieme dei conflitti. Questo consente di evitare passaggi inutili e velocizzare la ricerca.

Esempio:

Nel problema di colorazione dell'Australia, se il fallimento di una variabile V è dovuto a NT, il backjumping salta direttamente a NT, ignorando eventuali variabili intermedie (come SA, Q, NSW) non coinvolte nel conflitto.

L'implementazione richiede che ogni fallimento ritorni un insieme dei conflitti. Ogni livello della ricorsione riceve l'insieme dei conflitti dai suoi "*figli falliti*", **aggiunge la propria variabile e lo passa indietro**. Se l'insieme dei conflitti contiene la variabile corrente, si continua a tornare indietro. Altrimenti si prosegue con un altro valore.

Il backjumping può risultare meno utile quando si usano tecniche di inferenza potenti come MAC, perché molte inconsistenze vengono già rilevate anticipatamente.

Cap. 7: Agenti Logici

Il Capitolo 7 introduce gli *agenti logici*, una tipologia di agenti intelligenti che si basa sull'uso della **logica** e di una **base di conoscenza (KB)** per ragionare e pianificare le proprie azioni. A differenza degli agenti più semplici visti nei capitoli precedenti, che si limitano a reagire agli stimoli dell'ambiente in modo diretto, gli agenti logici sono in grado di **rappresentare esplicitamente lo stato del mondo** e **dedurre nuove informazioni** attraverso l'inferenza.

Questi agenti sono particolarmente utili in ambienti **complessi, incerti o sconosciuti**, dove le informazioni disponibili non sono complete o perfettamente osservabili. In questi contesti, l'agente non può semplicemente affidarsi alla percezione diretta, ma deve **ragionare** su ciò che sa e **inferire** ciò che non è immediatamente visibile, aggiornando la propria base di conoscenza man mano che riceve nuove informazioni.

Cap. 7.1 – Agenti basati sulla conoscenza

Il cuore di questi agenti è rappresentato dalla **base di conoscenza (KB)**, un insieme di informazioni organizzate in un **linguaggio formale** che l'agente utilizza per ragionare sul mondo e decidere le proprie azioni.

Il funzionamento di un agente basato sulla conoscenza segue un ciclo ben definito:

1. L'agente **percepisce** lo stato attuale dell'ambiente tramite i suoi sensori.
2. Queste percezioni vengono comunicate alla KB e utilizzate per **aggiornare la conoscenza** dell'agente (operazione **TELL**).
3. L'agente **interroga** la KB (**ASK**) per capire quale azione sia più opportuna da compiere, alla luce delle informazioni disponibili.
4. Infine, l'azione selezionata viene **eseguita** attraverso gli effettori dell'agente.

Al centro di questo processo c'è l'**inferenza**, il meccanismo logico che consente all'agente di **dedurre nuove informazioni** a partire da quelle già note. Ad esempio, nel mondo del Wumpus, l'agente deve essere in grado di concludere se una casella è sicura basandosi sugli indizi percepiti (come il fetore o la brezza).

Il comportamento dell'agente è codificato tramite due approcci:

- Uno di **natura dichiarativa**, definito da **regole generali** espresse nella KB. Ciò consente all'agente di affrontare situazioni nuove senza bisogno di istruzioni specifiche per ogni caso.
- Uno di **natura procedurale**, in cui i comportamenti desiderati sono codificati sotto forma di codice.

Spesso la Knowledge Base iniziale è definita attraverso un insieme di questi due approcci. Inoltre, questi agenti sono capaci di **apprendere**, ossia di acquisire nuova conoscenza man mano che esplorano l'ambiente e interagiscono con esso, aggiornando continuamente la propria base di conoscenza.

Cap. 7.2 – Il mondo del wumpus (NO)

Questo scenario viene utilizzato per illustrare in modo concreto come operano gli **agenti basati sulla conoscenza** all'interno di un ambiente incerto e parzialmente osservabile.

Il mondo del Wumpus è costituito da un sistema di **caverne**, suddivise in stanze connesse da passaggi. Le caratteristiche principali di questo ambiente sono:

- Un **Wumpus**, una creatura mostruosa che uccide chiunque entri nella sua stanza. Nella caverna è presente un solo Wumpus, il quale non può muoversi.
- Alcune stanze contengono **pozzi senza fondo**: chi ci entra precipita e muore.
- In una stanza è nascosto un **mucchio d'oro**.
- L'obiettivo dell'agente è **trovare l'oro**, raccoglierlo e **tornare sano e salvo al punto di partenza**, cercando di ottenere il punteggio più alto possibile.

Dal punto di vista tecnico, questo ambiente è definito come **discreto**, **deterministico**, **statico e a singolo agente**, ma soprattutto **parzialmente osservabile**: l'agente non ha una conoscenza completa dello stato dell'ambiente sin dall'inizio.

Per esplorare la caverna e prendere decisioni, l'agente si affida a **sensori** che gli forniscono informazioni indirette:

- **Fetore**: indica che il Wumpus si trova in una stanza adiacente.
- **Brezza**: segnala la presenza di un pozzo nelle vicinanze.
- **Scintillio**: si percepisce nella stanza in cui si trova l'oro.
- **Urto**: avviene quando l'agente sbatte contro un muro.
- **Grido**: si ode in tutta la caverna quando il Wumpus viene ucciso.

L'agente valuta le azioni attraverso questi sensi attraverso una **percezione** del tipo [Fetore, Brezza, none, none, none] e stabilisce se procedere verso la stanza è

un'azione sensata (se comporta un pericolo). Se la percezione restituisce come risultato 'OK' allora l'agente può procedere nella stanza senza morire.

L'agente può eseguire una serie di **azioni**:

- **Avanti**: si sposta nella stanza di fronte a lui.
- **GiraSinistra** e **GiraDestra** (di 90°): cambia la propria direzione.
- **Scocca**: lancia una freccia nella direzione attuale dell'agente; se la freccia incontra il wumpus nella sua corsa, lo uccide, altrimenti si rompe contro il muro.
- **Afferra**: raccoglie l'oro nella stanza attuale.
- **Esci**: esce dalla caverna passando per la stanza di partenza.

Il sistema di **punteggio** premia e penalizza le azioni dell'agente nel seguente modo:

- **+1000** per uscire dalla caverna con l'oro.
- **-1000** se l'agente muore (per mano del Wumpus o cadendo in un pozzo).
- **-1** per ogni azione eseguita.
- **-10** per l'utilizzo della freccia.

La **sfida principale** in questo ambiente è che l'agente, partendo da una conoscenza incompleta, deve **dedurre logicamente** dove si trovano i pericoli e quali stanze sono sicure. Questo implica l'uso sistematico della **base di conoscenza** e delle **tecniche di inferenza**, fondamentali per pianificare azioni efficaci e sopravvivere nel mondo del Wumpus.

Cap. 7.3 - Logica

Il capitolo 7.3 si concentra sul concetto di **logica**, la base fondamentale per la rappresentazione della conoscenza e il ragionamento degli **agenti logici**. Questa sezione non si limita a un singolo sistema logico, ma introduce concetti generali e universali, applicabili a diversi formalismi come la logica proposizionale o la logica del primo ordine, che saranno approfonditi più avanti.

I concetti principali introdotti in questa sezione sono:

1. Sintassi

La sintassi riguarda la **struttura formale** del linguaggio logico. Definisce le **formule ben formate** (o sentenze), ovvero le sequenze di simboli che rispettano le regole grammaticali del linguaggio. È ciò che consente di distinguere tra una formula valida (es. $\neg P_{1,2} \vee B_{1,2}$) e una sequenza priva di significato formale.

2. Semantica

La semantica attribuisce **significato** alle formule. In particolare, stabilisce **quando una formula è vera o falsa**, a seconda della configurazione del mondo in cui ci si trova. Ad esempio, nel mondo del Wumpus, la formula $P_{1,2}$ è vera solo se nella cella (1,2) c'è effettivamente un pozzo. La semantica è quindi essenziale per collegare le rappresentazioni formali (le formule) al mondo reale o simulato.

3. Mondi Possibili / Modelli

Un **modello** rappresenta un possibile stato del mondo. Ogni modello assegna valori di verità alle formule, e una formula è **vera in un modello** se descrive correttamente ciò che quel mondo rappresenta. La logica usa questi modelli per determinare quali formule sono vere in quali mondi.

Se una formula α è vera in un modello m , diciamo che **m è un modello di α** . Con la notazione $M(\alpha)$ si indica **l'insieme dei modelli di α** .

Ad esempio, una base di conoscenza può essere compatibile con più modelli (ossia, descrivere più mondi possibili), e ciascun modello ci aiuta a capire **quali stati del mondo sono ancora possibili** dato ciò che l'agente ha percepito finora.

4. Conseguenza Logica

La conseguenza logica è il **cuore del ragionamento logico**. Una formula β è una conseguenza logica di una base di conoscenza (si scrive $KB \models \beta$), se β è **vera in tutti i modelli in cui è vera anche KB** . In altre parole, se ciò che sappiamo (KB) è vero, allora anche β deve necessariamente esserlo. Questo concetto garantisce che ogni deduzione dell'agente sia **logicamente giustificata**.

5. Inferenza

L'inferenza è il **processo operativo** mediante il quale l'agente **deriva nuove informazioni** dalla propria base di conoscenza. Si fonda sull'uso di **regole d'inferenza** per dedurre conclusioni logicamente valide.

Se un algoritmo di inferenza i può derivare α dalla base di conoscenza, allora si esprime con la formula: $KB \vdash_i \alpha$.

Un algoritmo di inferenza è:

- **Corretto** se produce **solo** formule che sono effettivamente conseguenze logiche della KB .
- **Completo** se riesce a derivare **tutte** le formule che sono conseguenze logiche della KB .

In altre parole, un sistema ideale di inferenza dovrebbe essere sia corretto (*non dedurre nulla di falso*) sia completo (*non tralasciare nulla di vero*). Questo è cruciale per assicurare che un agente logico prenda decisioni **coerenti, affidabili e ben fondate**.

Cap. 7.4 – Logica proposizionale

La logica proposizionale è l'insieme di regole di **sintassi** e **semantica** necessarie a creare formule logicamente valide. Vedremo poi un algoritmo sintattico per l'inferenza logica che implementa la nozione semantica di **conseguenza logica**.

Sintassi:

Le **formule atomiche** rappresentano le **unità di base del linguaggio logico**.

Sono affermazioni semplici che non vengono ulteriormente scomposte all'interno del sistema logico. Ogni formula atomica può essere **vera o falsa**, ma non ha una struttura interna.

Sono solitamente indicate con **lettere maiuscole** come P, Q, R, eventualmente accompagnate da pedici numerici per rappresentare affermazioni relative a posizioni specifiche.

Ad esempio, nel mondo del Wumpus:

- W1,3 può significare "C'è un Wumpus nella casella (1,3)"
- P2,2 può significare "C'è un pozzo nella casella (2,2)"

Si possono **costruire formule più complesse** combinando formule atomiche (o anche formule già composte) attraverso i **connettivi logici**. Sono essenziali per esprimere relazioni logiche tra affermazioni.

I cinque connettivi logici principali sono:

- **Negazione ($\neg$)**: Indica il contrario di una formula.
Esempio: $\neg P$ significa "non è vero che P".
- **Congiunzione ($\wedge$)**: Indica che **entrambe le formule** devono essere vere. Esempio: $P \wedge Q$ significa "P e Q sono entrambe vere".
- **Disgiunzione ($\vee$)**: Indica che **almeno una** delle due formule è vera. Esempio: $P \vee Q$ significa "P o Q (o entrambe) sono vere".
- **Implicazione ($\Rightarrow$)**: Rappresenta una relazione condizionale.
Esempio: $P \Rightarrow Q$ significa "Se P è vera, allora anche Q deve essere vera". In questo caso, P è l'**antecedente**, Q il **conseguente**.

- **Bicondizionale (doppia implicazione) ($\Leftrightarrow$):** Indica che due formule sono **equivalenti**, cioè vere o false **insieme**.
Esempio: $P \Leftrightarrow Q$ significa "*P è vera se e solo se Q è vera*".

Semantica:

La semantica si occupa di definire il **significato** delle formule. Nel contesto della logica proposizionale, questo significato è dato dal **valore di verità** (*vero* o *falso*) di ogni formula in ciascun **mondo possibile**.

La semantica deve specificare se ogni **formula atomica** (simbolo proposizionale come P, Q, o nel mondo del Wumpus, come $P_{1,2}$ o $B_{1,1}$) è vera o falsa in un particolare **modello**.

Il valore di verità di una formula complessa, invece, viene determinato **ricorsivamente** in base ai valori di verità delle sue componenti (*formule atomiche o altre formule più semplici*) e alla definizione semantica dei **connettivi logici** utilizzati per combinarle.

La semantica dei connettivi logici è definita formalmente attraverso le **tavole di verità**. Queste tavole mostrano il valore di verità risultante per una formula complessa per ogni possibile combinazione dei valori di verità delle formule semplici che la compongono.

Costruzione di una Base di Conoscenza nel Mondo del Wumpus: (NO)

Costruiamo ora una KB destinata a un agente razionale che opera nel mondo del Wumpus. La base di conoscenza costruita in questa sezione è composta da due elementi principali:

1. Simboli Proposizionali (Formule Atomiche)

I simboli proposizionali rappresentano **affermazioni fondamentali** sullo stato delle caselle nel mondo del Wumpus. Questi simboli servono come unità di base della conoscenza rappresentata. Alcuni esempi tipici sono:

- $P_{x,y}$: C'è un **pozzo** nella casella (x,y).
- $W_{x,y}$: C'è un **Wumpus** nella casella (x,y).
- $B_{x,y}$: C'è una **brezza** percepita nella casella (x,y).
- $S_{x,y}$: C'è un **fetore** percepito nella casella (x,y).
- $L_{x,y}$: L'agente si trova nella casella (x,y).
- $OK_{x,y}$: La casella (x,y) è **sicura**.

Questi simboli permettono di rappresentare sia ciò che l'agente **percepisce**, sia ciò che **deduce** riguardo a pozzi, pericoli o sicurezza.

2. Formule Complesse (Regole Logiche)

Le formule complesse utilizzano i connettivi logici ($\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$) per **combinare i simboli atomici** e descrivere le **regole che governano l'ambiente fisico**.

Queste regole sono di due tipi principali:

- **Fatti iniziali noti**, *ad esempio*:
 $\neg P_{1,1}$ - Non c'è un pozzo nella casella di partenza.
- **Regole di correlazione tra percezioni e cause ambientali**, espresse tramite **bicondizionali** ($\Leftrightarrow$). Queste regole formalizzano il modo in cui le percezioni dell'agente sono causate da elementi del mondo, *ad esempio*:
 $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ - significa che nella casella (1,1) c'è **una brezza se e solo se** c'è un pozzo in (1,2) o in (2,1).

$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$ - Qui la brezza in (2,1) dipende dalla presenza di un pozzo in una delle caselle adiacenti.

Analogamente, anche il **fetore** ($S_{x,y}$) viene collegato in modo simile alla **presenza di un Wumpus** in una casella adiacente, secondo regole strutturalmente identiche.

Inferenza tramite Model Checking:

Trattiamo una **procedura semplice di inferenza** per la logica proposizionale, il cui scopo è determinare se una certa formula proposizionale, indicata con α , sia una **conseguenza logica** di una **base di conoscenza (KB)**. In termini formali, si tratta di verificare se $KB \models \alpha$.

Model Checking:

L'approccio presentato si basa su un algoritmo di **model checking**, che opera in modo esaustivo:

- **Enumerando esplicitamente tutti i possibili modelli**, cioè tutte le possibili combinazioni di valori di verità assegnabili ai simboli proposizionali coinvolti.
- Per ciascun modello in cui la **KB risulta vera**, si verifica se anche α è vera.
- Se α è vera in tutti questi modelli, allora si può concludere che α è una **conseguenza logica di KB**.

Il principale svantaggio dell'inferenza tramite model checking è di tipo **computazionale**. Lo **spazio dei modelli cresce esponenzialmente** con il numero di simboli proposizionali n . Di conseguenza, la complessità dell'algoritmo è $O(2^n)$. In termini pratici, ciò significa che anche per basi di conoscenza modeste, il numero di modelli da controllare può diventare rapidamente molto grande, rendendo l'inferenza **inefficiente o impraticabile** nel caso peggiore.

Cap. 7.5 – Dimostrazione di teoremi nella logica proposizionale

Introduciamo un **approccio deduttivo** all'inferenza logica, distinto dall'**approccio per enumerazione dei modelli** (*model checking*).

L'obiettivo resta lo stesso: determinare se una formula α è una **conseguenza logica** di una base di conoscenza **KB**, ovvero se $KB \models \alpha$.

Tuttavia, invece di analizzare tutti i modelli possibili, questo metodo si basa sulla **dimostrazione formale** di α a partire da **KB**, utilizzando **regole di inferenza**.

Si applicano dunque regole di inferenza per derivare nuove formule da KB. L'obiettivo è arrivare formalmente alla formula α , mostrando che essa **segue logicamente** la KB.

Può essere più **efficiente** in molti casi, specialmente quando la dimensione dello spazio dei modelli è grande. È la base per molti **sistemi di prova automatici e dimostratori di teoremi**.

Concetti fondamentali

Il capitolo illustra i concetti teorici necessari per comprendere e utilizzare la dimostrazione logica:

- **Equivalenza logica:**

Due formule α e β sono equivalenti se risultano **vere negli stessi modelli**, ciò si esprime con la notazione $\alpha \equiv \beta$.

- **Validità:**

Una formula è **valida** se è **vera in tutti i modelli** (*una tautologia*). Esempio:

$A \vee \neg A$ è una formula valida. Per definire la validità di un'espressione si utilizza il **teorema di deduzione**:

Date due formule α e β , $\alpha \models \beta$ se e solo se la formula $(\alpha \Rightarrow \beta)$ è valida.

- **Soddisfacibilità:**

Una formula è **soddisfacibile** se esiste almeno **un modello** in cui è vera. Il problema di decidere se una formula è soddisfacibile è noto come **SAT** (*satisfiability problem*) ed è **NP-completo**.

- **Dimostrazione per assurdo:**

Un metodo molto potente:

Per verificare se $\alpha \models \beta$, si può dimostrare che la formula $\alpha \wedge \neg \beta$ è **insoddisfacibile**, cioè **non esiste alcun modello** in cui sia vera. Se $\alpha \wedge \neg \beta$ non può essere vera in nessun modello, allora β è una conseguenza necessaria di α .

Regole di inferenza:

Le **regole di inferenza** sono strumenti formali che permettono di **derivare logicamente formule corrette** da formule già accettate come vere.

Esempi classici includono:

- **Modus ponens:** se $\alpha \Rightarrow \beta, \alpha$ allora si può inferire la formula β :
$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

(N.B. non è una frazione, significa che se vale la parte sopra allora si può inferire quello che sta sotto)

- **Eliminazione degli AND** ($\wedge$): data una congiunzione si può inferire uno qualsiasi dei congiunti:
$$\frac{\alpha \wedge \beta}{\alpha}$$

- **Regola di Risoluzione:** se applicata a un algoritmo di ricerca completo qualsiasi genera un **algoritmo di inferenza** completo, detto **PL- RISOLUZIONE**.

Per poter applicare l'algoritmo, tutte le formule devono essere espresse in **forma normale congiuntiva (CNF)**: $(A \vee \neg B) \wedge (B \vee C \vee \neg A)$.

Le **clausole** sono disgiunzioni di formule. La regola di risoluzione si applica a due clausole contenenti letterali complementari (A e $\neg A$). A queste viene applicata la **fattorizzazione**, ovvero vengono **rimossi i letterali complementari**:

clausole: $(A \vee B), (\neg A \vee C)$; *risolvente:* $(B \vee C)$;

Viene applicata **ricorsivamente** la regola di risoluzione a tutte le coppie di clausole. Ogni nuovo risolvente viene aggiunto all'insieme.

La **correttezza** si dimostra facilmente usando la **tabella di verità**: se entrambe le clausole originarie sono vere in un modello, allora anche la risolvente lo è. Dunque, ogni risoluzione mantiene la **correttezza logica**.

La **risoluzione si completa per refutazione**, cioè:

Se $KB \models \alpha$, allora $KB \wedge \neg \alpha \models \perp$ ($\perp$ è la *clausola vuota*).

La **clausola vuota** indica che l'insieme di clausole è **insoddisfacibile**, ovvero nessun modello lo rende vero. Ciò va in contraddizione, quindi α è **conseguenza logica** di KB.

Questo teorema è chiamato teorema di risoluzione ground: se un insieme di clausole è **insoddisfacibile**, la sua chiusura della risoluzione contiene la **clausola vuota**.

Clausole di Horn:

Una **clausola di Horn** è una **disgiunzione di letterali** (atomi o negazioni di atomi) in cui **al più un letterale è positivo**: $\neg P_1 \vee \neg P_2 \vee \dots \vee \neg P_n \vee Q$

Esistono due tipi di clausole di Horn:

1. **Clausole definite**: contengono **esattamente un letterale positivo**. In forma clausolare: $\neg P_1 \vee \neg P_2 \vee \dots \vee \neg P_n \vee Q$
Esempio: Se piove e ho l'ombrello, allora non mi bagno
 $Piove \wedge Ombrello \Rightarrow \neg Bagnato \rightarrow$ **clausola**: $\neg Piove \vee \neg Ombrello \vee \neg Bagnato$
2. **Clausole obiettivo**: non contengono letterali positivi, sono composti puramente da negazioni: $\neg P_1 \vee \neg P_2 \vee \dots \vee \neg P_n$
Spesso sono usate per rappresentare la **negazione di ciò che si vuole dimostrare**.
Esempio: Per dimostrare Q aggiungo la clausola obiettivo $\neg Q$, ovvero si parte assumendo che Q sia **falsa** per cercare una contraddizione.

Le clausole di Horn consentono inferenze molto più efficienti (*eseguibili in tempo polinomiale*) rispetto all'inferenza con formule generiche in CNF.

L'algoritmo di **Risoluzione Unitaria** funziona applicando la **risoluzione** solo quando almeno una delle clausole è **unitaria** (*ha un solo letterale*).

È **completo** per l'inferenza con clausole di Horn: se una formula è logicamente conseguente da una KB di clausole di Horn, allora verrà derivata con la risoluzione unitaria.

Metodi di inferenza con Clausole Definite:

Quando si lavora con **clausole definite**, un tipo particolare di clausole di Horn che contengono **esattamente un letterale positivo**, si possono applicare metodi di inferenza logica molto efficaci. In particolare, due algoritmi fondamentali sono la **concatenazione in avanti** e la **concatenazione all'indietro**.

Concatenazione in Avanti (*Forward Chaining*):

La concatenazione in avanti è un **metodo guidato dai dati**. Si parte dai **fatti noti** nella base di conoscenza e si usano le **regole di inferenza** (*cioè le clausole definite*) per cercare di derivare nuovi fatti. Ogni volta che le **premesse** di una regola risultano vere, allora la sua **conclusione** può essere aggiunta alla base di conoscenza come un nuovo fatto.

Questo processo continua finché:

- L'obiettivo che si vuole dimostrare viene effettivamente derivato
- OPPURE
- Non è più possibile generare nuove informazioni

Un semplice algoritmo chiamato **PL-CONSEGUE** illustra chiaramente questo approccio: l'algoritmo scorre tutte le clausole definite, verifica se le premesse sono soddisfatte, e se sì, aggiunge la conclusione come fatto.

Questo metodo è considerato **corretto e completo** per le clausole definite. Inoltre, è abbastanza **intuitivo per gli esseri umani**, poiché simula un tipo di ragionamento progressivo simile a quello che usiamo quotidianamente. Tuttavia, può diventare **inefficiente** se la base di conoscenza è molto ampia o se vengono prodotti molti fatti irrilevanti rispetto all'obiettivo.

Concatenazione all'Indietro (*Backward Chaining*):

La concatenazione all'indietro funziona al contrario: è un metodo **guidato dall'obiettivo**. Invece di partire dai fatti, parte da ciò che si vuole dimostrare. L'algoritmo cerca quindi nella base di conoscenza **regole** la cui **conclusione** corrisponde all'obiettivo. Se trova una regola adatta, allora cerca di dimostrare tutte le sue **premesse**, che diventano nuovi **sotto-obiettivi**.

Questo processo viene ripetuto ricorsivamente fino a quando:

- Tutti i sotto-obiettivi vengono dimostrati come fatti noti (*in tal caso l'obiettivo è dimostrato*), oppure
- Si arriva a un punto in cui non si riesce a dimostrare una delle premesse (*fallimento*).

Questo metodo è spesso più **efficiente** rispetto alla concatenazione in avanti, soprattutto quando la base di conoscenza è molto estesa e l'obiettivo è ben definito. Non esplora tutto lo spazio dei fatti possibili, ma si concentra solo su ciò che serve per dimostrare il risultato desiderato.

Un esempio pratico dell'uso della concatenazione all'indietro si trova nei linguaggi di programmazione logica come **Prolog**, dove le query vengono risolte proprio seguendo questa strategia.

Cap. 8: Logica di Primo Ordine

Il linguaggio della **logica del primo ordine** è costruito intorno agli **oggetti** e alle **relazioni**. Alcune relazioni hanno poi un solo valore per ciascun input e sono dette **funzioni**. Nella logica di primo ordine i **modelli** collegano simboli a valori di verità predefiniti e sono fondamentali per definire la semantica della logica di primo ordine.

Tali modelli sono costituiti da due elementi principali:

1. **Un dominio:** Questo è un **insieme di oggetti**. È un requisito fondamentale che il dominio **non sia vuoto**.
2. **Un'interpretazione:** Questa assegna un significato ai simboli del linguaggio logico rispetto agli oggetti nel dominio. In particolare:
 - Ad ogni **simbolo di costante** viene assegnato un oggetto nel dominio.
 - Ad ogni **simbolo di predicato** viene assegnata una relazione sugli oggetti nel dominio. Questa relazione è rappresentata come un insieme di **tuple di oggetti del dominio**. I predicati unari (*con un solo argomento*) sono rappresentati come sottoinsiemi del dominio.
 - Ad ogni **simbolo di funzione** viene assegnata una funzione totale che mappa tuple di oggetti del dominio ad oggetti del dominio

Simboli e interpretazioni:

La **sintassi** della logica di primo ordine, invece, si basa su **simboli** che hanno un significato preciso all'interno del dominio. Distinguiamo tre principali tipi di simboli:

1. **Simboli di costante:**
Questi simboli servono a rappresentare **oggetti specifici** nel dominio. Ogni costante denota un elemento preciso del mondo reale o immaginato.
2. **Simboli di predicato:**
I predicati esprimono **relazioni** o **proprietà** che coinvolgono uno o più oggetti. Ogni predicato ha una **arietà**, cioè un numero fisso di argomenti che richiede.
Esempio: Il predicato $Fratello(x, y)$ ha arietà 2 (è **binario**) e rappresenta la relazione di fratellanza tra due individui.
Il predicato $Persona(x)$ ha arietà 1 (è **unario**) e afferma che un certo oggetto è una persona.
3. **Simboli di funzione:**
A differenza dei predicati, i simboli di funzione **producono un oggetto** a partire da uno o più oggetti come input. Anche le funzioni hanno un'arietà che indica il numero di argomenti richiesti.
Esempio: Un esempio è $GambaSinistra(x)$, che prende una persona x e restituisce **la gamba sinistra di quella persona**. In questo caso, $GambaSinistra$ è una **funzione unaria**, perché prende un solo argomento.

Termini:

In generale, i **termini** sono utilizzati per rappresentare **oggetti del mondo** e si distinguono in due categorie principali:

- **Termini semplici:**

Questi sono i termini più elementari e possono essere:

- **Simboli di costante**, come “Riccardo” o “Giovanni”, che si riferiscono a oggetti specifici e ben identificati nel dominio.
- **Simboli di variabile**, come x , y , z , che rappresentano oggetti generici e possono assumere qualsiasi valore all'interno del dominio.

Le **costanti** sono quindi come nomi propri, mentre le **variabili** funzionano come segnaposto per oggetti qualsiasi.

- **Termini complessi:**

I termini complessi sono costruiti **applicando una funzione** a uno o più termini (*semplici o complessi*). La sintassi prevede che il nome della funzione sia seguito da una **parentesi con gli argomenti**, separati da virgole.

Esempio: Se “Padre” è un **simbolo di funzione unario** (cioè con arietà 1), allora l'espressione *Padre(Riccardo)* è un termine complesso valido.

Questo termine fa riferimento all'oggetto che è il padre di Riccardo, secondo quanto definito nel modello.

È fondamentale che il numero di argomenti forniti a una funzione corrisponda alla sua arietà. Quindi, una funzione binaria richiederà esattamente due termini come argomenti, e così via.

Formule atomiche:

Una **formula atomica** permette di asserire fatti. Chiamata anche **atomo**, è composta da un **simbolo di predicato** seguito da **una lista di termini tra parentesi**: *Fratello(Riccardo, Giovanni)* – Indica che Riccardo Cuor di Leone è fratello di Re Giovanni.

Le formule atomiche possono essere anche più complesse:

Sposato(Padre(Riccardo), Madre(Giovanni)) – Indica che il padre di Riccardo Cuor di Leone è sposato con la madre di Re Giovanni.

Infine, una formula atomica è **vera in un dato modello**, se la relazione a cui fa riferimento il simbolo di predicato è **verificata tra gli oggetti negli argomenti**.

Formule complesse:

Come nella logica proposizionale, si possono usare i connettivi logici per costruire formule complesse (o composte).

Esempio:

$\neg \text{Fratello}(\text{GambaSinistra}(\text{Riccardo}), \text{Giovanni})$

$\text{Fratello}(\text{Riccardo}, \text{Giovanni}) \wedge \text{Fratello}(\text{Giovanni}, \text{Riccardo})$

Quantificatori:

I **quantificatori** sono strumenti fondamentali della logica del primo ordine che ci permettono di esprimere affermazioni non solo su oggetti specifici, ma su **interi collezioni di oggetti** presenti nel dominio di un modello. Grazie ai quantificatori, possiamo formulare enunciati generali e astratti, andando oltre la semplice descrizione di singoli elementi identificati da costanti.

La logica del primo ordine utilizza due **quantificatori principali**:

1. Quantificatore Universale ($\forall$)

- Si legge come “*per ogni*” o “*per tutti*”.
- Viene impiegato per affermare che **una proprietà o una relazione vale per tutti gli oggetti** del dominio.
- La forma tipica è $\forall x P$, dove x è una **variabile** e P è una **formula** che descrive una proprietà o una relazione.

Ad esempio, la formula $\forall x (\text{Persona}(x))$ significa che *ogni oggetto del dominio è una persona*. Tuttavia, se il dominio comprende anche oggetti come una *corona*, questa formula risulterebbe **falsa**, poiché non tutti gli elementi del dominio soddisfano la proprietà.

In generale, **una formula universale è vera in un modello solo se la proprietà vale per ogni possibile oggetto a cui la variabile può riferirsi**.

2. Quantificatore Esistenziale ($\exists$)

- Si legge come “*esiste*” o “*c’è almeno un*”.
- Serve per esprimere che **almeno un oggetto nel dominio soddisfa una certa proprietà o relazione**.
- La sintassi è simile: $\exists x P$, dove x è una variabile e P una formula.

Ad esempio, la formula $\exists x (\text{SullaTesta}(x, \text{Giovanni}))$ si può leggere come “*esiste qualcosa sulla testa di Giovanni*”. Se, nel modello, la **corona** è effettivamente posizionata sulla testa di Giovanni, allora la formula sarà considerata **vera**.

In generale, **una formula esistenziale è vera se almeno un'istanza della variabile soddisfa la proprietà descritta nella formula.**

I quantificatori possono anche essere **combinati o annidati** per esprimere concetti più complessi. In questi casi, **l'ordine dei quantificatori è essenziale**, poiché può modificare radicalmente il significato dell'enunciato.

Esempio:

$\forall x \exists y \text{ Ama}(x, y)$ - significa "ognuno ama qualcuno".

$\exists y \forall x \text{ Ama}(x, y)$ - significa invece "esiste qualcuno che è amato da tutti".

Come si può notare, si tratta di due affermazioni profondamente diverse, nonostante utilizzino gli stessi simboli e predicati.

Notare come $\forall$ e $\exists$ **sono strettamente correlati fra loro attraverso la negazione**. Dire che tutti odiano i broccoli è come dire che non esiste nessuno a cui piacciono:

$\forall x \neg \text{Gradisce}(x, \text{Broccoli})$ è equivalente a $\neg \exists x \text{ Gradisce}(x, \text{Broccoli})$

Uguaglianza:

Si può costruire formule atomiche attraverso il simbolo di **uguaglianza**, affermando che **due termini fanno riferimento allo stesso oggetto**.

Esempio:

$Padre(Giovanni) = Enrico$

L'oggetto a cui si riferisce $Padre(Giovanni)$ è lo stesso a cui si riferisce $Enrico$. Si può anche utilizzare insieme alla negazione per specificare che due termini non fanno riferimento allo stesso oggetto:

Esempio:

$\exists y, x \text{ Fratello}(x, Riccardo) \wedge \text{Fratello}(y, Riccardo) \wedge \neg(x=y)$ – significa che Riccardo ha due fratelli diversi.

Semantica dei database:

Attraverso tre ipotesi si può stabilire una **semantica per i database**, dotata di un'**espressione logica più diretta**:

- **Ipotesi dei nomi unici** – ogni simbolo di costante fa riferimento a un oggetto distinto.
- **Ipotesi del mondo chiuso** – le formule atomiche sconosciute sono date per false.
- **Chiusura del dominio** – ogni modello contiene un numero di elementi del dominio non superiore agli elementi denotati dai simboli di costante.

Secondo questa semantica, con due oggetti ci sono 16 modelli possibili in tutto, molti meno rispetto alla semantica del primo ordine standard.

Asserzioni e query nella logica del primo ordine:

Per aggiungere formule alla KB si usa **TELL**. Queste formule sono dette **asserzioni**.

Esempio:

$TELL(KB, Re(Giovanni))$

$TELL(KB, \forall x Re(x))$

$\Rightarrow Persona(x)$

Per interrogare la KB si usa **ASK**:

Esempio:

$ASK(KB, Re(Giovanni))$

Le domande con ASK prendono il nome di **query**. Ogni query che è conseguenza logica della base di conoscenza dovrebbe avere risposta affermativa.

ASK dà una risposta che può avere come valori soltanto *true* e *false*. Per ottenere per quali **valori** è vera la formula si può utilizzare **ASKVARS**:

Quando sono presenti più valori si ottiene una **sostituzione** o **lista di legami**. ASKVARS funziona bene soltanto con KB costituite unicamente da clausole di Horn.

Cap. 9: Inferenza nella logica di Primo Ordine

Si può sfruttare una **strategia fondamentale per l'inferenza nella logica del primo ordine**: convertire i problemi della logica del primo ordine in problemi della **logica proposizionale**, in modo da poter applicare gli **algoritmi di inferenza già noti** per quest'ultima.

Per realizzare questa **riduzione**, è necessario introdurre **regole di inferenza specifiche per i quantificatori**. In particolare, si usano due regole principali:

1. Istanziamento Universale (UI)

Questa regola afferma che, se una formula è vera **per ogni oggetto del dominio** (cioè è *universalmente quantificata*), allora si può concludere che essa è vera anche per **qualsiasi oggetto specifico**.

Formalmente, da una formula come $\forall v \alpha$ si può derivare α , sostituendo la variabile v con **un termine ground** (cioè un termine che **non contiene variabili**, per esempio un nome specifico).

Esempio:

Se sappiamo che $\forall x \text{ Persona}(x)$ (cioè "Ogni x è una Persona"), allora possiamo concludere che $\text{Persona}(\text{Riccardo})$ è vera.

2. Istanziamento Esistenziale (Existential Instantiation)

Questa regola riguarda le affermazioni del tipo "esiste almeno un oggetto che...". Da una formula come $\exists v \alpha$ possiamo derivare α , sostituendo la variabile v con **una nuova costante** (detta **costante di Skolem**) che non compare altrove nella base di conoscenza.

Questa costante rappresenta **qualcosa che esiste**, anche se **non sappiamo esattamente cosa sia**.

Esempio:

Se abbiamo $\exists x \text{ HaUnaCorona}(x)$, possiamo inferire $\text{HaUnaCorona}(k)$, dove k è una nuova costante inventata (ad esempio, " k " può rappresentare un oggetto non identificato che ha una corona).

Proposizionalizzazione

Una volta applicate queste regole di **istanziamento**, si ottengono **formule prive di quantificatori**, chiamate **formule ground**, che possono essere trattate come formule della logica proposizionale.

Questo processo è chiamato **proposizionalizzazione**.

Per scrivere la regola di inferenza utilizziamo la **sostituzione**. Indichiamo con **SUBST**(θ, α) la sostituzione θ alla formula α , scrivendo la regola come:

$$\frac{\forall v \alpha}{SUBST(v/g, \alpha)}$$

Ad ogni variabile v sostituiremo il termine ground g .

L'obiettivo è trasformare una base di conoscenza (KB) espressa nella logica del primo ordine in una KB proposizionale **equivalente dal punto di vista inferenziale**, contenente tutte le formule ground necessarie.

Quando la KB include un simbolo di funzione, l'insieme di possibili sostituzioni in termini ground diventa **infinito**. Il **teorema di Herbrand** assicura che, se una formula segue logicamente la KB originale, allora esiste una dimostrazione che coinvolge solo un sottoinsieme finito della base proposizionalizzata.

A questo punto, **la logica del primo ordine diventa trattabile** con gli strumenti dell'inferenza proposizionale: si possono applicare algoritmi già studiati, come la risoluzione o il forward/backward chaining, e metodi di inferenza come il modus ponens, per derivare nuove conoscenze.

Si ricorre dunque a **procedure di inferenza che operano direttamente sulla logica del primo ordine**, senza bisogno di istanziare tutte le possibili formule ground.

Una tecnica fondamentale in questo approccio è il **lifting**, che consiste nel **generalizzare le regole della logica proposizionale** affinché possano essere applicate a formule contenenti variabili. Un esempio chiave di questa generalizzazione è il **Modus Ponens generalizzato**, che rappresenta la versione "*lifted*" del Modus Ponens per la logica del primo ordine.

Come funziona il Modus Ponens generalizzato:

1. Si considera una regola della forma: $P'_1 \wedge P'_2 \wedge \dots \wedge P'_n \Rightarrow Q'$,
dove le P'_i e Q' sono **formule atomiche** con variabili.
2. Si cercano nella KB dei **fatti noti** $P_1, P_2, \dots, P_n$ tali che ciascun P_i **unifichi** con il corrispondente P'_i nella premessa della regola.
3. L'**unificazione** è il processo che trova una **sostituzione** θ , cioè un insieme di assegnazioni che rendono identiche due formule logiche.
In questo contesto, si cerca una θ tale che, per ogni i , $SUBST(\theta, P'_i) = P_i$, ovvero la formula P'_i con la sostituzione θ diventa uguale al fatto noto P_i .

4. Se esiste una tale sostituzione θ che unifica **tutte le premesse**, allora possiamo **inferire** la formula Q' con la **stessa sostituzione applicata**, ottenendo $SUBST(\theta, Q')$.

Unificazione:

L'unificazione è il processo che, dato un insieme di espressioni (come termini o formule atomiche), cerca una **sostituzione** (denotata spesso con θ) che le renda **identiche** dal punto di vista sintattico.

La funzione **UNIFY**(x, y, θ) prende due espressioni x e y e una sostituzione iniziale θ e restituisce una sostituzione unificatrice o un indicatore di fallimento. La procedura UNIFY gestisce vari tipi di espressioni, inclusi simboli (costanti o variabili) e liste (che rappresentano argomenti di predicati o funzioni).

Esempi:

$UNIFY(Conosce(Giovanni, x), Conosce(Giovanni, Giacomina)) = \{x/Giacomina\}$

$UNIFY(Conosce(Giovanni, x), Conosce(y, Guglielmo)) = \{x/Giovanni, y/Guglielmo\}$

$UNIFY(Conosce(Giovanni, x), Conosce(y, Madre(y))) = \{y/Giovanni, x/Madre(Giovanni)\}$

$UNIFY(Conosce(Giovanni, x), Conosce(x, Elisabetta)) = \text{fallimento}$

L'ultima unificazione fallisce perché x non può assumere allo stesso tempo i valori Giovanni ed Elisabetta, il problema sorge unicamente perché le due formule usano lo stesso nome per la variabile (x).

Per evitare collisioni tra variabili con lo stesso nome, ma che appartengono a regole o clausole diverse, si utilizza una tecnica chiamata **standardizzazione separata** prima dell'unificazione. Si ridenominano semplicemente le variabili in modo che ciascuna clausola abbia variabili univoche.

Esempio:

$UNIFY(Conosce(Giovanni, x), Conosce(x, Elisabetta)) = \text{fallimento}$ – diventa:

$UNIFY(Conosce(Giovanni, x), Conosce(z, Elisabetta)) = \{x/Elisabetta, z/Giovanni\}$

L'unificazione non è usata solo nelle regole di inferenza "*lifted*". È necessaria anche per recuperare efficientemente fatti dalla base di conoscenza che unifichino una query a una premessa di una regola. Inoltre, è coinvolta nella gestione dell'uguaglianza nella logica del primo ordine.

Concatenazione in avanti:

Si può estendere il concetto di concatenazione in avanti composte da clausole definite anche a **clausole definite del primo ordine**.

Clausole definite del primo ordine:

Le clausole definite del primo ordine sono **disgiunzioni di letterali**, dei quali **esattamente uno è positivo**. In sostanza, una clausola è una formula atomica, oppure un'implicazione il cui antecedente è una congiunzione di letterali positivi e il conseguente è un singolo letterale positivo. I quantificatori esistenziali non sono ammessi e quelli universali sono lasciati impliciti.

Esempio:

$Re(x) \wedge Avido(x) \Rightarrow Malvagio(x)$

Avido(x) è interpretato come "Tutti sono avidi", il quantificatore universale è implicito

Un linguaggio costituito da clausole definite del primo ordine senza simboli di funzione è detto di tipo **Datalog**. L'assenza di funzioni rende l'inferenza più facile, motivo per il quale è utilizzato tipicamente nei database relazionali.

Semplice algoritmo di concatenazione in avanti (FOL-CA-ASK):

Partendo dai fatti noti, si fanno scattare tutte le regole le cui premesse sono soddisfatte, aggiungendo le conclusioni ai fatti noti. Questo processo si ripete finché non si trova una risposta o finché non è più possibile aggiungere nuovi fatti. Le ridenominazioni di fatti non costituiscono fatti nuovi (*Gradisce(x, Gelato)* e *Gradisce(y, Gelato)* hanno lo stesso significato).

Quando non è più possibile applicare alcuna inferenza nella KB, la situazione prende il nome di **punto fisso** del processo inferenziale.

FOL-CA-ASK è **corretto**, in quanto ogni inferenza è un'applicazione del Modus Ponens generalizzato, ed è inoltre **completo** per le basi di conoscenza di clausole definite.

Inefficienze dell'algoritmo di concatenazione in avanti:

Il semplice algoritmo di concatenazione in avanti, sebbene sia *corretto e completo per basi di conoscenza composte da clausole definite*, mostra delle **limitazioni significative** dal punto di vista dell'efficienza.

1. Riconsiderazione di regole e fatti a ogni passo

Una delle problematiche principali risiede nel fatto che l'algoritmo tende a riconsiderare ripetutamente le stesse regole e gli stessi fatti, anche quando non ci sono novità rilevanti. Questo comportamento **ridondante** porta a un **consumo elevato di risorse computazionali**, rendendo il processo poco scalabile su basi di conoscenza di grandi dimensioni.

2. Pattern matching

Un altro nodo critico è il *pattern matching*, ovvero l'operazione di **cercare fatti nella base di conoscenza che possano unificarsi con le premesse delle regole**. Quando una regola ha più premesse, il sistema deve trovare un numero di fatti uguale che si unifichino con ciascuna di esse utilizzando la **stessa sostituzione**. Questa ricerca può diventare rapidamente onerosa, soprattutto se fatta in modo ingenuo, perché implica l'esplorazione combinatoria di molteplici possibilità.

3. L'Ordinamento dei Congiunti

L'**ordine** in cui vengono abbinati i letterali congiunti nella premessa di una regola influisce notevolmente sull'efficienza. Trovare l'ordine ottimale per l'abbinamento (pattern matching) è un **problema NP-difficile** in generale, ma esistono buone **euristiche** che possono aiutare.

Una di queste è l euristica **MRV (Minimum Remaining Values)**; questa sceglie prima il letterale con minor possibilità di abbinamento, riducendo lo spazio di ricerca.

4. Fatti Irrilevanti

L'algoritmo semplice può inferire **fatti che non sono rilevanti per la query posta**. Sebbene per query generali (chiedere tutto ciò che è vero) possa essere accettabile, in altri casi l'inferenza di fatti non pertinenti spreca risorse.

Semplice algoritmo di concatenazione all'indietro (FOL-CI-ASK):

L'algoritmo semplice per la concatenazione all'indietro segue questi passaggi fondamentali:

1. Si comincia dall'obiettivo da dimostrare, oppure da una lista di congiunti (obiettivi parziali).
2. L'algoritmo cerca tra le regole della base di conoscenza quelle il cui **capo (head)** corrisponde all'obiettivo corrente.
3. Per verificare se una regola è applicabile, si usa il meccanismo di **unificazione**. È importante notare che le variabili nelle regole devono essere **standardizzate a parte**, cioè ridenominate, per evitare conflitti con quelle già presenti.

4. Quando si trova una regola il cui capo unifica con l'obiettivo, allora si passa a considerare le **premesse (body)** di quella regola come **nuovi sotto-obiettivi da dimostrare**.
5. Questo processo si ripete **ricorsivamente**, cercando di dimostrare ogni sotto-obiettivo.
6. Se tutte le premesse di una regola possono essere dimostrate — o se sono già presenti come fatti nella KB — allora si considera **dimostrato anche il capo della regola**, cioè l'obiettivo attuale.
7. L'intero processo termina con successo quando l'obiettivo iniziale è stato dimostrato, ovvero quando si giunge a fatti noti.

Dal punto di vista computazionale, l'algoritmo di concatenazione all'indietro può essere visto come una **ricerca in profondità** (*depth-first search*) nello **spazio degli obiettivi**. Questo approccio è generalmente efficiente, poiché si concentra solo su quelle parti della base di conoscenza che sono **pertinenti per raggiungere l'obiettivo**.

Tuttavia, il backward chaining può eseguire *computazioni ridondanti* se uno stesso sotto-obiettivo viene analizzato per diversi percorsi. Inoltre, con regole ricorsive l'algoritmo può incorrere in *cicli infiniti* senza completare. La ricerca in profondità, come sappiamo, può portare a *soluzioni subottimali*.

Applicazioni della concatenazione all'indietro, Prolog:

La **programmazione logica** è uno stile di programmazione strettamente connesso all'implementazione dell'**inferenza ideale**. L'obiettivo è quello di avvicinarsi all'ideale espresso dalla **formula di Kowalski**, secondo cui:

Algoritmi = Logica + Controllo

A supporto di questa visione, viene presentato **Prolog**, il linguaggio per eccellenza della programmazione logica. I programmi scritti in Prolog sono costituiti da un insieme di **clausole definite**, che rappresentano conoscenza e regole inferenziali.

Una particolarità stilistica importante di Prolog, che lo distingue dalla notazione abituale della logica del primo ordine, è la convenzione secondo cui:

- **Le variabili iniziano con una lettera maiuscola**
- **Le costanti e i predicati iniziano con una lettera minuscola**

Dal punto di vista dell'esecuzione, un programma Prolog si comporta essenzialmente come un motore di inferenza che applica la **concatenazione all'indietro** (*backward chaining*), effettuando una **ricerca in profondità** nella base di conoscenza. In questo contesto, **l'ordine delle clausole** all'interno del programma **ha un impatto diretto** sull'ordine di esplorazione della ricerca.

Un altro aspetto caratteristico è il modo in cui Prolog tratta le **funzioni aritmetiche**: a differenza della logica pura, Prolog non le valuta direttamente come formule logiche, ma le considera istruzioni da **eseguire attivamente** nel corso della computazione.

Risoluzione nella Logica di Primo Ordine:

La **risoluzione**, proprio come per la logica proposizionale, si basa su una regola di inferenza applicabile a basi di conoscenza espresse in **forma clausolare**. Nella logica di primo ordine è **completa per refutazione**, ovvero, può essere usata per dimostrare che una certa formula **non può essere falsa**, dato che la sua negazione porta a una contraddizione.

Per verificare che una base di conoscenza (KB) implica una formula α , assume che α sia falsa e cerca di dimostrare che **$KB \wedge \neg\alpha$ sia insoddisfacibile**

(L'obiettivo dell'algoritmo di risoluzione è quindi quello di derivare la clausola vuota, che rappresenta una contraddizione logica).

Per usare la risoluzione, è necessario che la base di conoscenza sia espressa in **forma normale congiuntiva** (CNF), ovvero le formule devono essere **congiunzioni di clausole**, dove ogni clausola è una disgiunzione di letterali.

Nella FOL, i **quantificatori universali sono impliciti**, mentre i **quantificatori esistenziali** devono essere eliminati tramite un processo chiamato **Skolemizzazione**.

Fasi per convertire una formula in CNF

1. **Eliminare implicazioni e doppie implicazioni**, che vanno riscritte usando solo negazioni, congiunzioni e disgiunzioni.
2. **Spostare le negazioni all'interno** della formula, applicando le leggi di De Morgan.
3. **Standardizzare le variabili**, cioè rinominare le variabili quantificate per evitare ambiguità.
4. **Portare tutti i quantificatori all'esterno** della formula (prenex form)
5. **Skolemizzazione**: si rimuovono i quantificatori esistenziali sostituendo le variabili con costanti di Skolem o funzioni di Skolem, in base al contesto.
6. **Distribuire la congiunzione sulla disgiunzione**, per ottenere la struttura CNF.
7. **Appiattire le espressioni**, cioè eliminare le nidificazioni superflue.
8. **Rimuovere i quantificatori universali**, poiché sono sottintesi.

Cap. 10: Rappresentazione della conoscenza

L'ingegneria ontologica si occupa di come rappresentare la conoscenza in modo efficace, specialmente in domini complessi, dove è importante utilizzare rappresentazioni **flessibili** e **generali**. In contesti più semplici o "*mondi giocattolo*", la scelta della rappresentazione non è così critica, ma in applicazioni reali diventa fondamentale.

Per gestire questa complessità, si introducono concetti astratti e strutturati attraverso quella che si chiama **ontologia (o rete semantica)**, che funge da quadro generale per definire concetti, proprietà e relazioni. Spesso si usano anche dei "**segnaposto**" (*placeholders*) per elementi che possono essere meglio definiti o aggiunti in un secondo momento.

Una delle sfide più importanti nell'ingegneria ontologica è trovare un giusto equilibrio tra la capacità di catturare le **generalizzazioni principali** e quella di gestire le **eccezioni**. Questo equilibrio è cruciale per costruire un'ontologia efficace e robusta.

Inoltre, costruire un'ontologia completa non è semplice: il lavoro coinvolge aspetti sia **filosofici** sia **computazionali**, come la difficile questione di decidere cosa esiste e cosa no nel dominio che si vuole rappresentare.

Categorie e oggetti:

Come si possono organizzare gli oggetti nel mondo, per una corretta rappresentazione della conoscenza?

Organizzazione degli oggetti:

Gli oggetti sono organizzati in **categorie**. Sebbene per gli esseri umani sia relativamente facile formare categorie e definire le proprietà dei loro membri, è un compito complesso per l'ingegneria ontologica.

Tassonomia:

Una tassonomia è una **gerarchia di categorie**. La logica del primo ordine consente di esprimere queste relazioni gerarchiche, come una categoria che è un sottoinsieme di un'altra (*SottoinsiemeDi*). Le tassonomie sono rappresentabili come alberi.

Relazioni tra oggetti e categorie:

$BB \in \text{PalloniDaBasket}$ – indica che BB è un pallone da basket

$\text{PalloniDaBasket} \subset \text{Palloni}$ – indica che la sottocategoria palloni da basket fa parte della categoria dei palloni

Definizioni delle categorie:

Due categorie sono dette **disgiunte** quando non hanno istanze in comune. Esse costituiscono una **decomposizione esaustiva** quando tutte le istanze della sovracategoria sono presenti in almeno una delle due sottocategorie. Infine, si dice che costituiscono una partizione quando sono **disgiunte e costituiscono una decomposizione esaustiva**.

Gli oggetti possono avere degli aspetti specifici che vanno rappresentati nel modo corretto

Composizione fisica:

La relazione di **composizione** tra oggetti stabilisce come un oggetto può far parte di un altro. Viene usato il predicato **ParteDi(oggetto, categoria_o_entità_superiore)**.

Misure:

Bisogna poter rappresentare gli attributi quantitativi degli oggetti, come altezza, peso o costo. Il concetto di **unità di misura** dà un senso ai valori numerici, permettendo di fare conversioni tra diverse unità, pur trattando la stessa misura. È insensato invece assegnare valori numerici a qualità soggettive come la bellezza.

Oggetti; cose e roba:

Esiste una distinzione fondamentale tra **oggetti discreti e numerabili** ("cose") e **sostanze o collezioni non facilmente numerabili individualmente** ("roba"), come l'acqua o la sabbia. La "roba" è spesso descritta in termini di composizione o misurazione, mentre le "cose" sono oggetti distinti.

Le proprietà degli oggetti possono essere **intrinseche** (come il colore) o **estrinseche** (come il nome o il peso). Entrambi i tipi ("cose" e "roba") rappresentano le generalizzazioni principali in questa parte dell'ontologia.

Eventi:

Gli eventi sono azioni o accadimenti che avvengono nel mondo ne modificano lo stato nel tempo.

Per modellare questi cambiamenti, si introduce il concetto di **fluenti**, ovvero aspetti del mondo la cui verità può variare nel tempo. Viene anche fatta una distinzione importante tra **azioni**, che sono compiute da un agente, e **eventi**, che semplicemente accadono, anche senza un agente diretto.

Tempo:

Il tempo può essere descritto sia come **punti** precisi che come **intervalli** estesi. Per una descrizione precisa, si utilizzano unità di misura standard e si definiscono relazioni temporali tra eventi o intervalli, come ad esempio:

- *Precede* (un intervallo avviene prima di un altro),
- *Segue* (uno viene dopo l'altro),
- *Durante*, *Sovrapposti*, *FineContemporanea*, *Uguale* e *Consecutivi*.

Per rappresentare eventi in relazione al tempo, si introducono predicati come:

- *Accade(evento, istante)*
- *Inizia(evento, istante)*

Fluenti e oggetti:

Qui si affronta il modo in cui alcuni oggetti possono essere considerati **fluenti generalizzati**, ossia oggetti le cui proprietà cambiano nel tempo. Ad esempio, uno Stato può cambiare il numero delle sue regioni, o una persona può assumere o perdere una carica pubblica.

Per descrivere queste variazioni si usano predicati come:

- *Proprietà(oggetto, tempo)*

oppure funzioni che restituiscono valori diversi a seconda del momento temporale considerato. Questo approccio consente di catturare le dinamiche del mondo reale, mantenendo la possibilità di generalizzare e strutturare la conoscenza in modo robusto.

Esempio:

T(Uguale(Presidente(Stati Uniti), GeorgeWashington), Inizio(AD1790), Fine(AD1790))

Relazioni Tra ontologie:

Elenchiamo i tipi di relazione possibili tra ontologie:

1. **Identical:** *O1* e *O2* sono la stessa ontologia (esempio: *un'ontologia e la sua copia in un disco mirror*).
2. **Equivalent:** condividono vocabolario e assiomatizzazione ma sono espresse in linguaggi differenti (esempio: *SKOS e RDF*).
3. **Extension:** *O1* estende *O2* quando tutti i simboli definiti in *O2* sono preservati in *O1* insieme alle loro proprietà e relazioni ma non vale il viceversa.
4. **Weakly-Translatable:** siano *Osource* e *Odest* due ontologie, è possibile tradurre espressioni *Osource* in espressioni *Odest* con perdita di informazione.
5. **Strongly-Translatable:** *Osource* è *strongly-translatable* in *Odest* quando:
 - Il suo vocabolario è totalmente mappabile in quello di *Odest*
 - L'assiomatizzazione di *Osource* vale in *Odest*
 - Non c'è perdita di informazione – Non si introducono inconsistenze
6. **Approx-Translatable:** *Osource* è *Approx-Translatable* in *Odest* quando è *WeaklyTranslatable* e possono essere introdotte delle inconsistenze. Tipicamente questo accade perché alcuni concetti che dovrebbero corrispondere l'uno con l'altro risultano affini ma non identici.

Azioni:

Molte applicazioni dei sistemi di inferenza concernono il **decidere quali azioni eseguire**, tipicamente per raggiungere un obiettivo. **Pianificare** significa costruire una sequenza di azioni per soddisfare un certo fine.

In generale, in un certo stato solo un sottoinsieme delle azioni sarà applicabile e di queste solo una sarà applicata.

Situation Calculus:

La rappresentazione e il ragionamento su azioni si basa spesso sul **Situation Calculus**, una rappresentazione logica che identifica i concetti base di:

- **Azione:** qualcosa che viene compiuto e influenza il mondo.
- **Situazione:** stati derivanti dall'esecuzione di qualche azione.
- **Fluente:** proprietà che può cambiare valore.
- **Predicato atemporale (eterno):** sono funzioni o predicati il cui calcolo non è influenzato dalle azioni.

Tramite **Do(Azioni, S)** un agente può fare **proiezione**, ovvero ragionare sugli effetti delle azioni. In particolare, può:

- **Verificare** se un corso di azione attraversa situazioni che godono di determinate proprietà.
- **Pianificare** un corso di azione, ovvero individuare quale sequenza di azioni permette di raggiungere una situazione che gode di una specifica proprietà.

Nella logica, un'azione è descritta da assiomi di applicabilità e assiomi di effetto:

- **Assioma di Applicabilità:**

$$\forall \text{ params, s } \text{Applicable}(\text{Action}(\text{params}), \text{s}) \Leftrightarrow \text{Precond}(\text{params}, \text{s})$$

Definisce che un'azione può essere applicata in una situazione se e solo se valgono le condizioni a destra.

- **Assioma di Effetto:**

$$\forall \text{ params, s } \text{Applicable}(\text{Action}(\text{params}), \text{s}) \Rightarrow \text{Effects}(\text{params}, \text{Result}(\text{Action}(\text{params}), \text{s}))$$

Specifica gli effetti di un'azione sulla situazione in cui è eseguita. *Effects* è una **formula** vera nello stato risultante dall'esecuzione dell'azione.

Result è una **funzione** che denota lo stato in cui si va eseguendo l'azione in s.

Dalla conoscenza di stato iniziale, assiomi di applicabilità e assiomi di effetto **NON** è possibile derivare tutti i fatti che ci aspettiamo.

Non è detto che un'azione debba necessariamente modificare dei fluenti, introduciamo un terzo assioma:

- **Assioma di Stato Successore:**

Azione applicabile $\Rightarrow$ (fluente vero nella situazione risultante $\Leftrightarrow$ (l'azione lo rende vero $\vee$ (or) era vero e l'azione non l'ha reso falso))

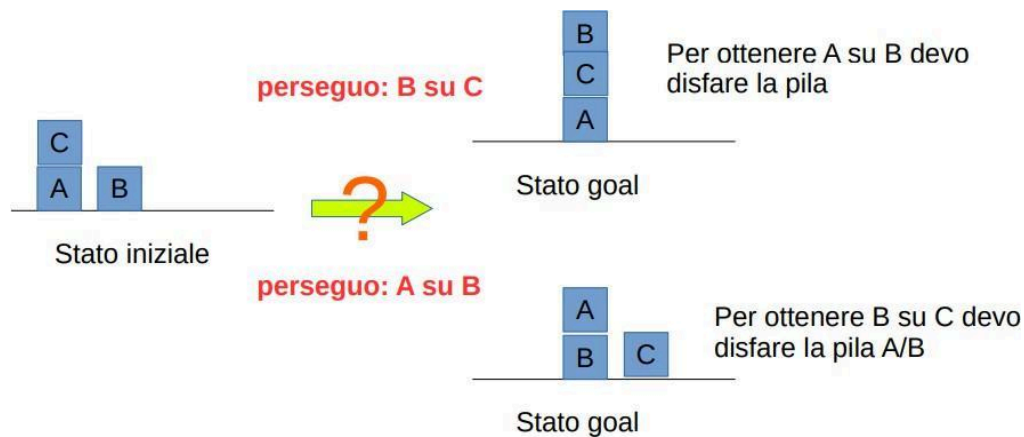
Questo assioma esprime il modo in cui le **situazioni si evolvono** l'una dall'altra a seguito dell'esecuzione delle azioni. In particolare, dice quali parti di una situazione sono ereditati dalla precedente.

Infine il Situation Calculus è caratterizzato da assiomi in grado di riconoscere che **azioni con nomi diversi sono differenti** e che per ogni nome di azione, **due utilizzi di esso sono identici se e soltanto se gli argomenti lo sono a loro volta**.

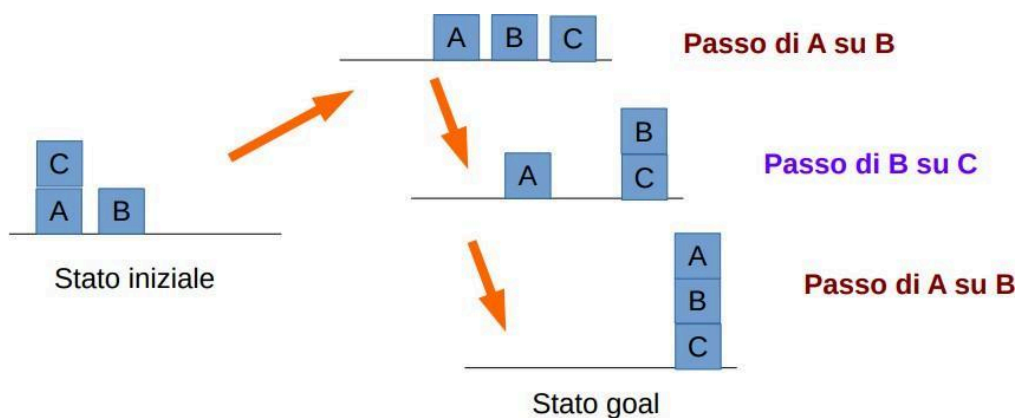
Pianificazione:

Un approccio tipico alla pianificazione è quello di scomporre l'obiettivo in **sotto obiettivi** e conseguirli uno alla volta.

Tuttavia, per l'**Anomalia di Sussman**, non sempre è possibile sequenzializzare il perseguimento degli obiettivi, alcune volte infatti perseguire un sottogoal può disfare i passi effettuati per raggiungerne un altro.



Una soluzione all'Anomalia di Sussman è fare interleaving dei passi di soluzione.



Apprendimento (*machine learning*)

L'apprendimento parte dalla **classificazione** degli elementi rilevati dall'agente. Le classi non esistono a livello generico, ma vengono definite soltanto **a seconda del fine** per il quale si vuole costruire un predittore.

L'**apprendimento** consiste nel raccogliere dati e costruire un **modello** (*una rappresentazione astratta*) che permetta di associare in modo corretto nuove istanze alla classe (o alle classi) di appartenenza.

Generalmente quello di un agente è detto **apprendimento supervisionato**, ovvero possiede già esempi a cui è già associata la classe corretta.

Il **learning set**, invece, è la collezione di dati usati per svolgere il compito di apprendimento. I dati sono divisi in **tuple**, contenenti il nome dell'istanza e tutte le informazioni su di essa, compresa la classe di appartenenza.

Modelli predittivi e modelli descrittivi:

Il **modello predittivo** viene usato per predire la classe di appartenenza di istanze ignote in fase di apprendimento. Il **modello descrittivo**, invece, viene usato come strumento esplicativo che permette di evidenziare quali caratteristiche distinguono le diverse categorie.

Valutazione dei modelli appresi:

La valutazione della bontà di un modello appreso è fatta sulla base del comportamento di classificazione corretto o sbagliato su questi dati.

Matrice di confusione:

		classe predetta	
		Classe 1	Classe 2
classe reale	Classe 1	f11	f12
	Classe 2	f21	f22



Errori !! **Predizioni corrette**

La matrice di confusione confronta le occorrenze delle previsioni nel modello con le etichette reali (la verità). Ciò permette di vedere dove l'apprendimento sbaglia e dove indovina.

L'**accuratezza** misura quante sono le occorrenze di predizioni corrette su tutti i test, mentre l'**error rate** quante sono i test che danno errore.

Matrice dei costi:

		classe predetta	
		Classe 1	Classe 2
classe reale	Classe 1	c1	c2
	Classe 2	c3	c4



Costi degli errori !! **Costi delle valutazioni corrette**

La **matrice dei costi** è un'estensione della matrice di confusione che permette di tenere conto non solo della correttezza di una previsione, ma anche di *quanto costa* ciascun tipo di errore o correttezza. Viene mantenuta la classificazione con i costi minori.

Memorizzazione delle istanze:

La **memorizzazione delle istanze** (*instance-based learning* o *lazy learning*) è un approccio all'apprendimento automatico in cui il modello non costruisce una generalizzazione esplicita dei dati durante la fase di training. Invece, **memorizza direttamente i dati di addestramento** e prende decisioni **solo quando è necessario fare una previsione**.

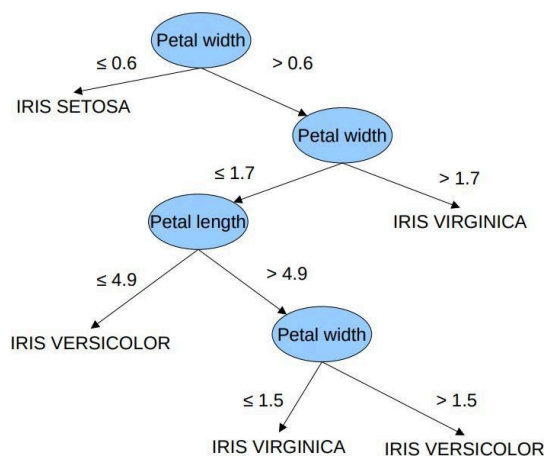
Algoritmi di apprendimento diversi producono modelli di tipo diverso come:

- **Alberi di decisione:** albero
- **Sistemi a regole:** insiemi di regole if-then
- **Reti neurali:** matrici di numeri
- **Apprendimento per rinforzo:** distribuzioni di probabilità e matrici di numeri
- ...

Alberi di decisione:

Sono strumenti di supporto alle **decisioni** che usano modelli strutturati ad albero, comunemente utilizzati per esempio per la definizione di strategie mirate al conseguimento di un goal.

In un albero di decisione, i **nodi interni** rappresentano i **test**, mentre le **foglie** rappresentano le **decisioni**.



Algoritmo di Hunt:

L'**algoritmo di Hunt** è un famoso algoritmo per l'apprendimento di alberi di decisione.

L'albero viene costruito procedendo ricorsivamente e suddividendo il learning set in sottoinsiemi via via più "puri". Il generico passo di suddivisione di un nodo dell'albero esegue questi passi:

Dati:

D_t = sottoinsieme del learning set associato al nodo t

$y = \{y_1, y_2, \dots, y_c\}$ = insieme delle etichette che identificano le classi

Passo 1: se tutte le istanze in D_t appartengono alla stessa classe y_t , ovvero **se tutti gli esempi nel nodo appartengono alla stessa classe**, allora il **nodo diventa una foglia** che viene **etichettata** dalla classe y_t delle sue istanze.

Passo 2: si valuta ogni attributo (feature) per determinare quale suddivide (**split**) meglio i dati secondo le classi.

Passo 3: si sceglie un **attributo** fra quelli che descrivono le istanze, **si produce un nodo figlio per ogni possibile valore dell'attributo**. A ciascun nodo figlio si associa uno specifico valore e si associano ad esso anche quelle istanze, già associate al padre, per le quali l'attributo assume il valore corrispondente al nodo.

Passo 4: si applica l'algoritmo ricorsivamente a ciascun sotto-nodo finché tutti gli esempi in un nodo appartengono alla stessa classe, oppure non ci sono più attributi su cui dividere.

Nota 1: se una certa combinazione di valori non è rappresentata da nessuna istanza, questa sarà associata alla classe di default (se esiste).

Nota 2: se tutte le istanze associate a un nodo sono identiche come tuple ma corrispondono a classi differenti (*non-determinismo*), il nodo non può essere scisso. Diventa una foglia che ha associata la classe più rappresentata.

Split:

Lo **split** o **suddivisione**, è il processo col quale si sceglie l'attributo che separa meglio i dati secondo le classi. Gli attributi su cui effettuare uno split sono selezionati in modo da **massimizzare** una qualche misura di riferimento.

Esistono quattro tipi di split:

- **Split su attributi binari:**
Nello split binario un nodo avrà due figli con due valori diversi.
- **Split su attributi nominali:**
Anche qui, uno split può essere **binario** oppure **multivalore**. Nel primo il nodo avrà due figli, uno corrispondente a un valore e l'altro all'insieme dei valori rimanenti. Il secondo invece avrà tanti figli quanti i possibili valori dell'attributo.
- **Split su attributi ordinali:**
Si possono avere split binari o multivalore, tuttavia il raggruppamento dei valori deve rispettare l'ordinamento.
- **Split su attributi continui:**
In questo caso si cerca di identificare un insieme di valori v_i per l'attributo A in questione e la produzione di una serie di test $v_i \leq A < v_{i+1}$

Con quale criterio si scelgono gli alberi?

Generalmente, alberi compatti sono preferibili rispetto ad alberi con più test.

Per il **Rasoio di Occam**, a parità di assunzioni, la spiegazione più semplice è da preferire.

Dunque, gli split preferibili, sono quelli che producono figli la cui estensione prevede minor confusione (*con grado di purezza maggiore*).

Misure per determinare la bontà di uno split:

Per quantificare la "confusione" o "impurità" di un nodo, si usano diverse misure:

- **Entropia**
- **Gini**
- **Errore di classificazione**

La **base di queste misure** è la probabilità $p(i | t)$ che un elemento estratto casualmente dall'insieme D_t (associato al nodo t) sia di classe i .

$$\text{Entropia}(t) = - \sum_{i=0}^{c-1} p(i|t) \log_2 p(i|t)$$

Dove c rappresenta il numero di classi e $p(i | t)$ la probabilità che un elemento appartenga alla classe i .

L'entropia misura il grado di **purezza** o **imprevedibilità**. Se un insieme è perfettamente **puro** l'entropia è 0, mentre se un insieme ha il **grado massimo di confusione**, l'entropia è 1.

Calcolo del guadagno (gain):

La misura del risultato di uno split è rappresentabile con la formula:

$$\Delta = I(\text{parent}) - \sum_{j=0}^k \frac{N(v_j)}{N} = I(v_j)$$

Dall'**impurità del nodo genitore** viene sottratta la **media pesata delle impurità dei nodi figli**. Di solito la misura dell'impurità è scelta in modo tale da minimizzare l'impurità / massimizzare il guadagno.

Information gain:

Per **information gain** si intende una misura del guadagno ottenuta usando l'**entropia** come valore dell'impurità dei nodi:

$$\Delta = \text{entropia}(\text{parent}) - \sum_{j=0}^k \frac{N(v_j)}{N} = \text{entropia}(v_j)$$

Misure del grado di confusione come l'entropia, favoriscono attributi che hanno **molti valori diversi**, rispetto ad attributi con pochi valori alternativi. Un identificatore univoco, dove ogni nodo conterrà un'unica istanza, non è un attributo significativo per lo split.

Overfitting:

Quando un modello appreso è troppo specifico avviene un fenomeno detto **overfitting**. Ciò avviene specialmente con grandi alberi.

L'overfitting nello specifico può essere dato da presenza di **noise**, ovvero istanze classificate in modo errato, oppure dalla **manca di esempi**, ovvero non rappresenta tutti i casi significativi.

Consideriamo la costruzione di un albero di decisione: ad ogni iterazione occorre individuare un attributo su cui effettuare il test. Un attributo **viene preso in considerazione** se il guadagno che dà supera una soglia minima.

Spesso la procedura di costruzione è **greedy**: cerca di massimizzare il guadagno.

Il modello ideale è quello che produce il **minor errore di generalizzazione possibile**. Come poter approssimare il modello ideale quando si ha a disposizione solo un insieme di esempi di learning?

Ebbene, per il principio del **Rasoio di Occam**, tra modelli a parità di errore, quelli più **semplici** vengono preferiti.

Pruning:

Per semplificare l'albero si usa la potatura (*pruning*). Esistono due tipi di potature:

- **Pre-Pruning:** la costruzione dell'albero si interrompe prima che l'albero sia completo, attraverso una regola di terminazione più restrittiva.
- **Post-Pruning:** prima si costruisce l'albero poi si potano alcuni rami, trasformando alcuni nodi interni in foglie.

Esistono, dunque, delle regole per il pruning.

Pre-pruning:

Non si esegue lo split se il gain è al di sotto di una certa soglia. Un vantaggio è che evita l'overfitting dei dati, tuttavia uno svantaggio è che la scelta della soglia può portare all'underfitting.

Post-pruning:

Questo approccio consiste nel tagliare rami da un albero fatto crescere finché non si hanno più guadagni.

Possibili strategie:

- Sostituisco un sottoalbero col suo cammino usato più di frequente.
- Sostituisco un sottoalbero con una foglia la cui classe corrisponde alla classe **maggiormente rappresentata** nelle foglie dell'albero rimosso.

Bisogna sottolineare che tagliare rami costruiti significa che il tempo trascorso a costruirli è stato sprecato.

Reti Neurali:

Le reti neurali si ispirano al modo in cui i neuroni agiscono e interagiscono. Fra i modelli più diffusi è presente il **Perceptron**.

Perceptron:

Un **perceptron** è un elemento computazionale, dotato di una piccola memoria in grado di calcolare una **funzione di attivazione** in esso strettamente codificata: **$Y = f(\text{net})$**

Questa funzione è calcolata su una composizione dei valori in ingresso, opportunamente pesati.

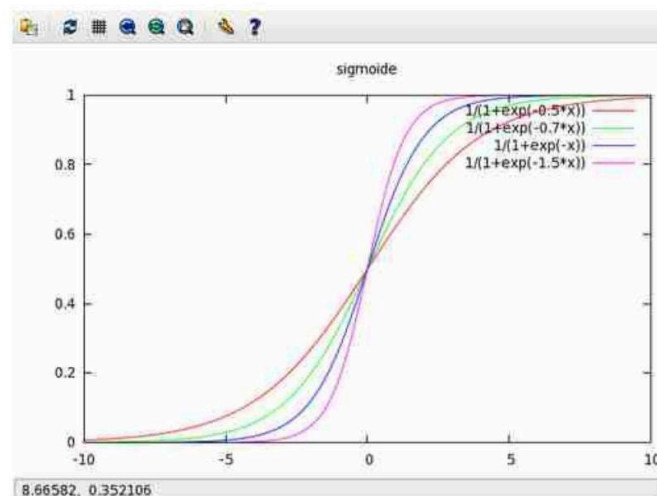
Il primo tipo di funzione di attivazione è detta **funzione gradino**, essa è una **combinazione lineare** dei valori in ingresso:

$$\text{net} = \sum_{i=1}^n w_i X_i$$

Molto più vantaggiosa è la **funzione sigmoide**, la quale è derivabile:

$$Y = f(\text{net}) = \frac{1}{1 + e^{-\alpha(\text{net} - \theta)}}$$

Dove θ rappresenta la **soglia** o **bias** (parametro preimpostato), mentre α controlla la pendenza.



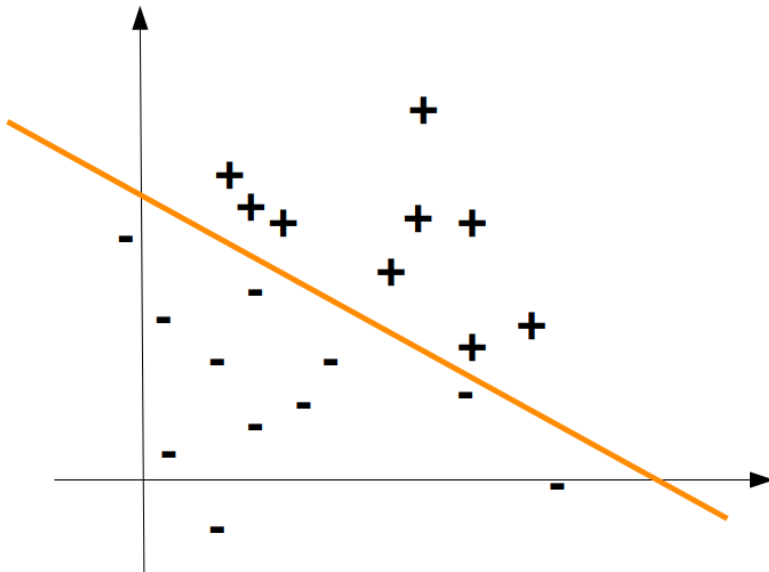
Nota: al crescere del parametro α la sigmoide tende alla funzione gradino

Passata forward:

La **passata forward** (o *forward pass*) è la **prima fase** nel funzionamento di una rete neurale, e serve a **calcolare l'output** della rete dati gli input. Più precisamente, la passata forward è una sequenza in cui il perceptron (il neurone):

- Riceve un **input** (**vettore di caratteristiche (feature)** dell'istanza **x**)
- Pesa ciascun input con dei **pesi (w)**
- Somma i prodotti di ciascuna **x** con il suo peso **w** e aggiunge un **bias (b)**
- Passa il risultato a una **funzione di attivazione** (*restituisce 0 o 1 se supera una certa soglia o meno*).

Il risultato Y della passata forward permette di ottenere una **classificazione**, un dato può ricadere nella categoria 0 oppure nella categoria 1.



Un perceptron codifica un **test lineare**, in cui ciò che accade sopra l'iperpiano codificato dai pesi fa attivare il neurone. Ciò che sta al di sotto non fa attivare il neurone.

Quello che si trova sopra l'iperpiano appartiene alla classe obiettivo, mentre al di sotto stanno le istanze negative.

I **pesi** sulle connessioni in ingresso definiscono la posizione e la pendenza dell'iperpiano nello spazio in cui sono definiti gli input. I **pesi** caratterizzano i neuroni e **costituiscono la conoscenza del neurone**.

Il Perceptron è adatto specialmente a svolgere **compiti di tipo numerico** e consente di **risolvere problemi separabili linearmente**. È necessario ricordare che i pesi, la conoscenza del Perceptron, sono **persistenti** e vengono modificati durante la sua esecuzione.

Passata backward:

La **passata backward** (*backpropagation*) segue subito dopo la passata forward e aggiorna i pesi in base all'errore tra output previsto e reale:

$$w_j^{(k+1)} = w_j^k + \alpha * (d - o) x_j$$

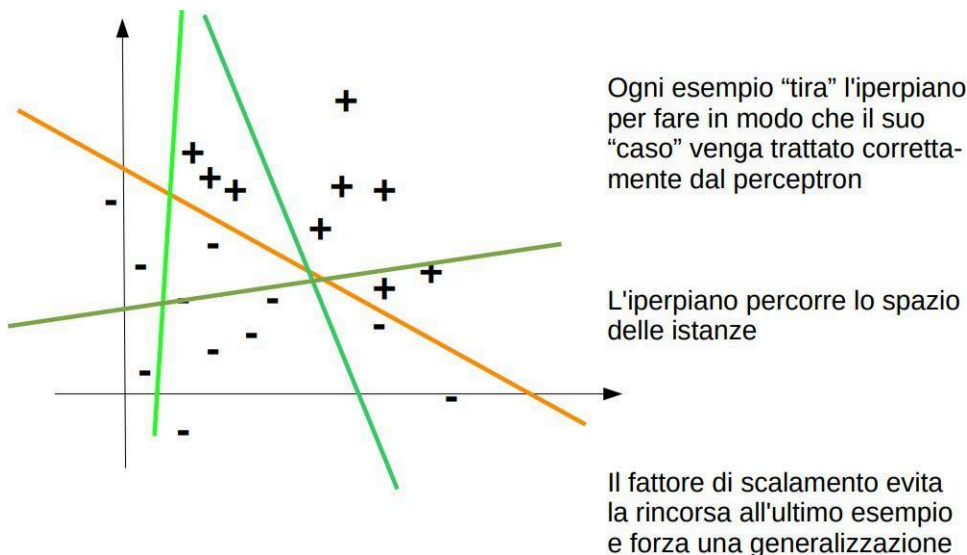
Si utilizza l'errore atteso ($d-o$) come differenza tra il valore desiderato e quello ottenuto. Il peso w sulla connessione fra l'**input j-esimo** e il **neurone** viene aggiornato sommandogli l'**errore moltiplicato per un fattore di scalamento α** e per il valore della **componente j-esima** dell'input precedentemente elaborato.

L'errore ($d-o$) è l'informazione che permette di aggiornare i pesi e rendere il comportamento futuro del Perceptron più preciso.

È stato dimostrato da Novikoff che se il problema è linearmente separabile, l'apprendimento converge alla soluzione, altrimenti no.

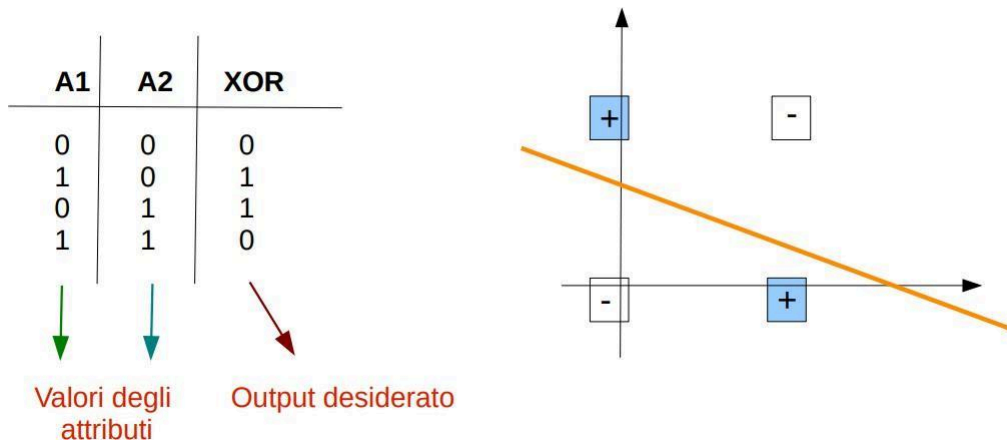
L'apprendimento percorre tutto il learning set e ad ogni iterazione:

- La prima istanza del learning set viene passata forward nel perceptron, che produce un output.
- L'errore di questo viene usato in una passata backward che ne modifica i pesi
- Poi si passa all'istanza successiva



Limiti del Perceptron:

Un esempio semplicissimo mostra i limiti del Perceptron, l'**or esclusivo (XOR)**.



Non basta un iperpiano per trovare la soluzione corretta allo XOR. Già due potrebbero funzionare. In realtà l'insieme di più iperpiani è in grado persino di circoscrivere la soluzione nel piano.

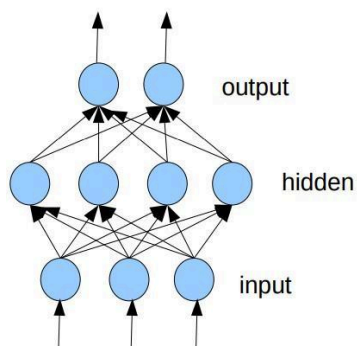
Reti neurali:

Una rete neurale è un **approssimatore universale di funzioni**, avente natura distribuita. È costituita da un insieme di neuroni, collegati tra di loro secondo una **topologia**. I neuroni possono implementare funzioni diverse. Possono essere software oppure hardware.

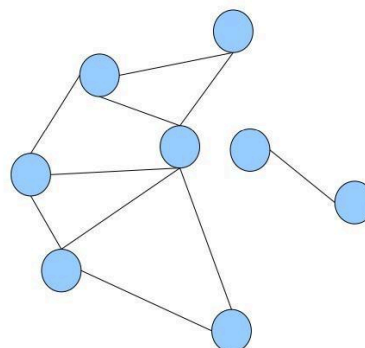
La topologia di una rete è il **modo in cui sono organizzati i neuroni** (nodi) e le loro connessioni.

Esempi

A strati (layered)



A vicinato

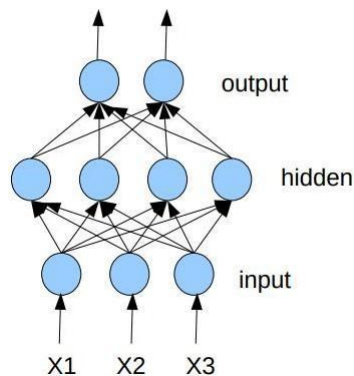


Multi-layer-perceptron (MLP):

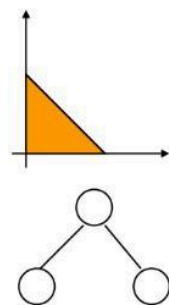
Il **multi-layer perceptron** è un modello di rete neurale con **topologia a strati**, feed-forward (ovvero il flusso di calcolo è rivolto ad una sola direzione). I neuroni di **input** implementano la funzione identità, mentre i neuroni **hidden** sono perceptron che utilizzano la sigmoide. I neuroni **output** combinano i risultati dei neuroni hidden.

MLP e classificazione

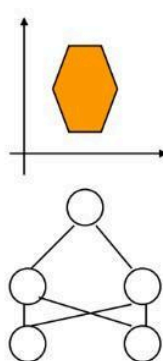
Codifica delle classi



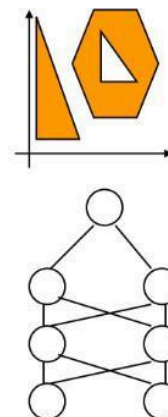
1. Se il problema è riconoscere le istanze della classe X, basta un neurone di output. Se si attiva si ha il riconoscimento.
2. Se il problema è distinguere le istanze di classe X da quelle di classe Y, basta un neurone di output. Si associa l'attivazione alla classe X.
3. Se il problema è distinguere tra 3 classi occorrono almeno 2 neuroni: il problema è infatti rappresentare tutti i numeri fino a 3 in modo binario. Spesso però si usa un neurone di output per ogni classe. Il neurone corrispondente alla classe giusta deve attivarsi, gli altri rimanere a zero.



Il primo layer
traccia dei confini



Il secondo layer
costruisce delle forme



Il terzo layer
crea forme qualsivoglia
complesse

Una rete neurale di tipo MLP **impara in modo supervisionato** inducendo la matrice dei pesi a partire da un insieme di esempi (**batch**).

Per ogni istanza vengono effettuate:

- **Passata in avanti (forward pass)**: l'istanza viene sottoposta all'MLP che la elabora e produce un risultato;
- **Passata all'indietro (backpropagation)**: l'**errore** viene utilizzato per modificare i **pesi** partendo dalle connessioni più vicine ai neuroni di output e procedendo a ritroso

Il learning set viene **elaborato dalla rete molte volte**, ogni passata dell'intero learning set è detta **epoca di apprendimento**.

Discesa del gradiente:

La backpropagation utilizza la formula di **discesa del gradiente** per stimare l'**errore atteso** di ciascun peso w :

$$\Delta w_{ji} = - \lambda \frac{\partial E(\bar{w})}{\partial w_{ji}}$$

Ogni peso w_{ji} (su una connessione dal neurone j -mo al neurone i -mo) viene modificato sottraendo la derivata parziale dell'errore rispetto al peso stesso.

Il gradiente indica quanto bisogna modificare ciascun peso per raggiungere un errore minore.

L'**errore** viene calcolato esclusivamente per i neuroni del **layer di output**, ma qual è la formula dell'errore? Quale segnale utilizzare per modificare i pesi fra tutti i neuroni degli strati precedenti?

$$E = 1/2 \sum_{i=1}^p \|t_i - y_i\|^2$$

p = numero dei neuroni di output

t_i = valore desiderato per il neurone di output i -esimo

y_i = valore prodotto dal neurone di output i -esimo